

TECHNICAL REPORT · SOURCE AUDIT

从视频压缩到 Codec-Native 多模态模型

OneVision-Encoder、codec-video-prep、
LLaVA-OneVision-2 与 Mage-VL 技术报告

论文逐节核对 · 官方源码追踪 · 调用链与数据流复原

2026 年 8 月 17 日

面向视频编码、多模态模型与系统工程读者

目录

以下为章节级目录。Word 中点击条目可直接跳转。

摘要

阅读约定

审计材料与版本

全文路线图

第一章 视频压缩从哪里节省了数据

第二章 I、P、B 与三种不同的 GOP

第三章 从码流中能读出哪些视觉线索

第四章 从完整帧到稀疏视觉 token

第五章 OneVision-Encoder 把 codec 稀疏性变成视觉原语

第六章 codec-video-prep 如何把码流变成模型输入

第七章 LLaVA-OneVision-2 把稀疏视觉接进通用 MLLM

第八章 Mage-ViT 与 Mage-VL 走向跨 codec 和流式交互

第九章 把五层系统连成一条数据流

第十章 如何评测、复现和部署

第十一章 后续技术可能怎样发展

第十二章 结论

附录 A 术语速查

附录 B 仓库版本与关键入口

附录 C 关键审计发现清单

附录 D 主要论文与资料

附录 E 一页式总结

摘要

视频模型最昂贵的浪费，往往发生在视觉编码器看到第一枚 token 之前。普通长视频管线会等间隔抽取若干完整 RGB 帧，再把每个画面切成密集 patch。相邻帧大部分区域几乎没有变化，视觉编码器仍会为背景、静止物体和重复纹理反复计算。传统视频编码器已经为同一个问题工作了数十年。它用帧内预测、帧间预测、运动信息、预测残差、变换、量化和熵编码，把可预测的重复内容压到很少的比特里。Codec-native 视觉模型的核心想法，是复用这些编码结构来决定哪些时空区域值得变成视觉 token。

这条路线远比“读取运动矢量和残差”复杂，实际跨越四层边界。第一层是视频标准与 FFmpeg 解码器，决定哪些信号真实存在、处于什么粒度。第二层是预处理选择器，负责抽帧、分组、评分、预算分配、patch 选择和画布打包。第三层是视觉编码器，负责理解被打散的时空 token。第四层是多模态语言模型，负责把视觉证据用于问答、时间定位、字幕生成或流式交互。任一层的坐标、顺序、掩码或预算语义出错，都可能让一套看上去合理的算法在实际训练中失效。

本文先建立对视频压缩的整体认识，再解释 OneVision-Encoder 如何把 codec 对齐稀疏性变成统一视觉输入。随后对 `codec-video-prep` 的两条真实流水线做逐函数追踪，继续核对 LLaVA-OneVision-2 与 Mage-VL、Mage-ViT 的论文设计和官方代码边界。报告将“论文提出的思想”“官方代码实际执行的路径”“后续工作继承或改变的部分”分开书写。源码审计还发现数处需要警惕的实现差异，包括公开 CLI 的采样枚举不匹配、README 中位置张量格式与代码不一致、元数据里的解码后端被硬编码、LLaVA 适配器的目标画布参数未真正约束输出数量。它们不会否定技术路线，却会直接影响复现和结果解释。

阅读约定

全文使用三种证据标签。

- **论文思想** 论文明确提出的算法、假设或实验结论。
- **代码实现** 在审计提交中能够沿调用链确认的真实行为。

- **继承与修改** 后续工作保留了什么，又在哪些环节改变了选择器、编码器、训练课程或部署方式。

技术结论后会给出论文节号、图表号，或源码文件与函数。源码路径均相对相应仓库根目录。仓库持续更新时，函数名比行号更稳定，因此正文以文件和函数为主，附录再记录审计提交。

审计材料与版本

对象	本文用途	主要依据
HEVC Overview	建立混合编码、CTU、预测、变换、滤波和并行工具的基础	Section II、IV，Figure 1
OneVision-Encoder	codec 对齐视觉 token 化和统一视觉编码器	Section 2，Figure 4，Appendix Section 8
codec-video-prep	patched FFmpeg、采样、分组、Top-K、画布与位置数据的真实工程链	官方仓库源码与用户提供的技术说明
LLaVA-OneVision-2	自适应逻辑 GOP、codec 训练混合和通用 MLLM 接入	Section 2、8，Figure 2 及官方仓库
Mage-VL / Mage-ViT	跨 codec 视觉编码、滚动视频窗口和主动发言	Section 3、4，Table 2 及官方仓库

审计边界 本文解释所给论文和公开仓库在审计提交上的行为。论文实验所用的内部数据、训练集清洗细节和未公开基础设施无法由仓库完整还原。凡是源码不能证明的环节，正文会明确标为论文设计或合理推断。

全文路线图

1. 先弄清视频为何能被压缩，以及压缩器到底计算了什么。
2. 再区分 I、P、B 图片，随机访问点，编码 GOP 和预处理逻辑组。
3. 然后理解 packet size、运动矢量、残差与 bitcost 各自能说明什么。
4. 接着进入 OneVision-Encoder，解释稀疏 patch、画布和 3D 位置编码。
5. 再沿 codec-video-prep 的入口走到 patched FFmpeg、选择器和输出张量。
6. 最后核对 LLaVA-OneVision-2 与 Mage-VL 如何继承、改造并部署这套思想。

第一章 视频压缩从哪里节省了数据

1.1 原始视频为何巨大

一张 1920×1080 的 8 bit RGB 图像约有 622 万字节。每秒 30 帧时，未经压缩的数据量接近 187 MB/s，一分钟超过 11 GB。实际视频常用 YCbCr 4:2:0，单个像素平均约 1.5 字节，同样条件下仍有约 93 MB/s，一分钟约 5.6 GB。网络视频能缩小到每秒几兆比特，依赖的是画面中大量可预测的结构。

可利用的冗余主要分布在几个方向。

- 同一帧里，相邻像素和纹理彼此相似。
- 相邻帧里，大块背景没有变化，移动物体常能从参考帧平移得到。
- 变换后的系数通常集中在少量低频或显著分量上。
- 编码语法和量化系数的出现概率不均匀，可以用更短的码字表示常见事件。

这几个方向分别对应帧内预测、帧间预测、变换量化和熵编码。视频编码器的工作可以概括为两步。先构造一个尽可能准确的预测，再只编码预测仍解释不了的部分。

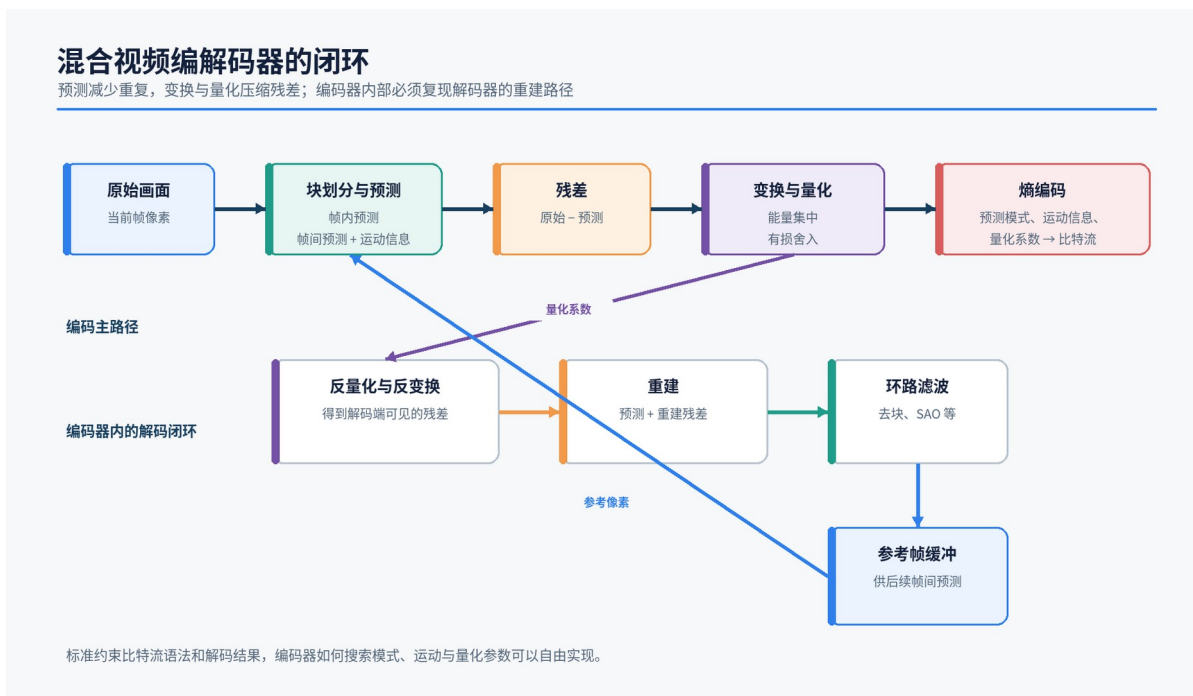
1.2 标准规定什么，编码器决定什么

HEVC 等视频标准主要规定比特流语法和解码过程。只要生成的码流合规，解码器就应得到规定的重建画面。标准并不要求编码器用同一种搜索算法。运动搜索范围、块划分策略、率失真权衡、快速剪枝和码率控制都可以由编码器开发者决定。

这一区分对 codec-native 模型很重要。模型读到的运动矢量、块类型和 bitcost，是某个具体编码器在特定参数下做出的选择。它们受到画面内容影响，也受到编码 preset、码率、GOP 长度、参考帧数量和量化参数影响。同一段视频经不同编码器处理，信号不会逐块相同。

【论文依据】HEVC Overview Section II.A 与 Figure 1。论文明确说明标准给出语法和解码过程，编码器设计保留较大自由度。

1.3 混合编码闭环



图中的上半部分是编码主路径。当前块先经过帧内或帧间预测，原始像素减去预测像素得到残差。残差经过变换和量化，随后与预测模式、运动信息等语法一起进入熵编码器。最终比特流不需要携带完整当前画面，只需携带解码器复现预测与重建所需的信息。

下半部分解释了一个容易忽略的事实。编码器内部也要执行反量化、反变换、重建和环路滤波。后续帧引用的是解码端可得到的重建画面，不能引用编码器手中的无损原图。若两端参考图不同，预测误差会沿时间不断累积，解码结果很快失控。因此编码器内部含有一个与解码器一致的重建闭环。

1.4 从 RGB 到 YCbCr 4:2:0

RGB 直接描述红、绿、蓝三个颜色分量。视频编码常先变换到 YCbCr。Y 近似表示亮度，Cb 与 Cr 表示两个色度差分量。人眼对亮度边缘更敏感，对色度细节相对宽容，于是常见的 4:2:0 采样会保留完整亮度分辨率，并在水平和垂直方向各把色度采样减半。

以 2×2 像素块为例，4:2:0 保留四个 Y 样本，只保留一个 Cb 和一个 Cr 样本。总计六个 8 bit 样本，平均每个像素 12 bit，也就是 1.5 字节。它本身已经比 24 bit RGB 节省一半数据，后续预测与量化还会继续压缩。

Codec-native 前端最终经常仍把被选区域还原成 RGB 画布，因为现成视觉处理器和 ViT 接口通常接收 RGB。压缩域信号负责选择，RGB patch 负责提供可识别的外观。两者承担不同角色。

【论文依据】HEVC Overview Section IV.A。

1.5 CTU、CU、PU 与 TU

现代视频编码不是把整帧当成一个预测单位。HEVC 先把画面划分为编码树单元 CTU，常见边长为 64，也允许 16 或 32。每个 CTU 可以递归分成编码单元 CU。CU 再通过预测单元 PU 描述采用何种预测区域，通过变换单元 TU 描述残差怎样变换。

可以把四个概念理解成四个问题。

结构	回答的问题	对 codec-native 选择的影响
CTU	这一大片区域怎样作为编码树根节点	patched HEVC 常在这里记录粗粒度 bitcost
CU	是否继续细分，采用何种编码模式	反映局部复杂度和编码器搜索结果
PU	用哪个参考区域与运动矢量做预测	运动信息通常按这类预测块产生
TU	残差在哪个区域做何种变换	残差能量和系数成本受其边界影响

视觉模型的 patch 网格通常与编码块网格不对齐。OneVision-Encoder 使用 14×14 像素 patch，Mage-ViT 使用 16×16。HEVC 的 CTU、PU 和 TU 大小可变。因此工程实现必须把编码块分数投影或分配到一个规则像素网格，再聚合到 ViT patch 或 2×2 patch block。

【论文依据】HEVC Overview Section II、IV。

1.6 帧内预测和帧间预测

帧内预测只使用同一画面中已经重建的邻近像素。例如一块平坦墙面可以从左侧或上方边缘延伸得到。HEVC 提供多种方向模式，编码器选择误差和码率更合适的一种。

帧间预测从一张或多张参考画面取出矩形区域。运动矢量描述当前预测块相对参考位置的位移，参考索引指出使用哪张参考图。若物体只是平移，编码器只需传输运动和少量残差。若发生遮挡、形变、光照变化或新物体出现，预测无法完整解释当前区域，残差会增加。

运动矢量是编码器为了重建服务的块位移。它与光流相似，却没有逐像素运动场的语义保证。低纹理区域可能存在多个同样合理的匹配，编码器会偏向码率更低的向量。摄像机整体移动也会让背景获得很大的运动幅度。直接把运动幅度当作重要性，只能得到一种廉价代理。

1.7 残差、变换与量化

残差定义为原始块减去预测块。残差仍是像素域二维信号。变换把它映射到频率样式的系数，使能量更容易集中。平滑变化常落到少数低频系数，细纹理和锐利边缘会产生更多高频分量。

量化把连续或高精度系数除以步长并舍入。步长越大，更多小系数会变为零，码率下降，失真上升。量化是现代有损视频编码里最直接的不可逆步骤。反量化无法恢复被舍掉的小数和归零系数，只能得到近似残差。

当 codec-native 方法使用 residual energy 时，常见做法是计算一个块内亮度残差绝对值、平方和或相关统计。它能突出预测困难区域，也会对噪声、细碎纹理和全局亮度变化敏感。分数高只代表编码难度增加，并不自动等于下游问题相关。

1.8 熵编码与 bitcost

熵编码利用符号概率不均匀继续缩短码流。HEVC 的 CABAC 会把二值化语法元素、上下文概率和算术编码结合起来。编码模式、运动差分、变换系数显著性、系数幅度等都会消耗比特。

bitcost 可以在不同层面定义。粗略做法是使用一帧压缩包的字节数。更细做法是在解码某个宏块、CTU 或语法区域前后读取熵解码器的 bit position，再取差值。codec-video-prep 的 patched FFmpeg 走的是后者。它在 H.264、HEVC 与 VP9 解码路径中记录粗网格和 4x4 子网格成本，然后通过 AVFrame.opaque_ref 暴露给 C++ 扩展。

一个区域的 bitcost 高，通常意味着编码器为了描述模式、运动或残差花费了更多语法比特。它把多类编码困难压缩成一个标量，计算成本又低。与此同时，它仍受码率控制和编码配置影响。子网格里一部分语法成本只能按覆盖范围分摊，因此“真实总比特”与“精确语义归因”需要分开理解。

1.9 环路滤波与参考画面

分块预测和量化容易在块边界产生不连续，也会形成振铃等伪影。HEVC 在重建后应用去块滤波和样点自适应偏移 SAO。滤波后的图像进入解码画面缓冲区，成为后续帧间预测的参考。

这一点解释了为何只读取运动矢量和残差还不够。下游视觉模型最终需要识别物体外观，仍要取得重建像素。Codec-native 前端通常通过解码器同时获得结构信号和 RGB 像素，再把被选中的像素 patch 拼成画布。

1.10 率失真优化

编码器需要在更清晰和更省比特之间选择。典型目标可以写成

$$J = D + \lambda R$$

其中 D 是重建失真， R 是预计码率， λ 控制两者权重。一个预测模式即使像素误差更小，若需要大量运动和系数比特，也可能输给稍模糊但码率更低的模式。

这意味着 codec 信号天然带有“在当前编码目标下值得花多少比特”的含义。它比纯像素差更接近编码器的综合决策，却仍不包含用户问题。未来可学习选择器的关键任务，是把编码率失真先验与任务相关性结合，而不是完全抛弃任一方。

第二章 I、P、B 与三种不同的 GOP

2.1 I、P、B 是编码预测类型

I 图片的编码区域可以独立于其他画面重建。P 图片允许从过去的参考图做单向帧间预测。B 图片允许使用两个时间方向的参考图，也可以包含更灵活的参考组合。实际标准中这些名称与 slice 类型相关，一张 P 或 B 图片内部仍可能出现帧内编码块。

日常说法常把 I 帧称为“完整帧”，这在理解码流时容易造成误会。I 图片也经过变换、量化和熵编码，解码后得到的是有损重建像素。它的预测依赖更少。P/B 图片同样可以完整解码为一张画面，码流里只是没有逐像素保存整幅图。

2.2 IDR、CRA 与容器同步样本

随机访问需要一个可以安全开始解码的位置。H.264 常见 IDR，HEVC 还定义 CRA 等随机访问图像。IDR 会切断对更早参考图的依赖，便于干净跳转。CRA 允许更灵活的解码刷新语义，附近某些前导图片需要按规则处理。

MP4 容器的 `stss` 表列出同步样本。播放器可从这些样本开始随机访问。`codec-video-prep` 优先解析这个表来得到需要规避的 frame id，失败后再调用 `ffprobe` 读取关键帧时间戳。同步样本是容器和解码意义上的入口，与镜头切换处最有代表性的画面没有必然等价关系。

【源码依据】`codec_selector/codec_patch_gop/video_probe.py` 中的 `mp4_keyframe_frame_ids` 与 `ffprobe_keyframe_frame_ids`。

2.3 显示顺序和解码顺序

B 图片可能引用显示时间上更晚的参考图。编码器必须先发送并解码那个未来参考，再回头重建 B 图片。因此容器中的解码顺序可能与观众看到的显示顺序不同。时间戳通常包含 PTS 与 DTS，分别服务显示和解码调度。

预处理代码若只把 packet 索引当作绝对画面时间，容易把时序错位。稳妥做法是通过 ffprobe 和解码器给出的帧信息建立显示帧号，再用 fps 或 PTS 生成时间标签。codec-video-prep 的公开链接把采样目标表示为 frame id，LLaVA 适配层再将来源帧号换成时间戳文本。

2.4 编码 GOP

编码 GOP 是编码时形成的一段预测与随机访问结构。它受关键图片间隔、B 图片模式、参考列表和场景切换策略控制。典型示意可以写成 I B B P B B P，真实依赖图往往比字符串更复杂。

它在视频写入码流时已经决定。后续预处理无法通过“重新分组若干 frame id”改变原码流的预测依赖，除非重新编码视频。

2.5 LLaVA-OneVision-2 的逻辑 GOP

LLaVA-OneVision-2 论文根据 P/B 包大小形成时间能量，再用累计配额、最小和最大组长以及局部低谷搜索建立自适应段。论文仍称这些段为 GOP。它们服务采样和视觉预算，不会改写视频编码结构。为避免混淆，本文称其为自适应逻辑 GOP。

2.6 codec-video-prep 的 readiness 组

公开单视频链先得到候选帧列表，再根据 block 分数阈值、时间覆盖桶、最小和最大帧数、边际收益动态结束当前组。这是另一种预处理逻辑组。组首帧通常作为完整 anchor，其余帧竞争有限的 2x2 block 预算。

同一个“GOP”词，实际可能指三件事

报告中把码流结构、论文的逻辑分桶、工程的 readiness 分组分开讨论

① 编码 GOP

编码时形成



决定预测依赖、随机访问和解码顺序

② LLaVA-OV2 逻辑 GOP

预处理时按 P/B 包能量切段



决定候选帧密度与组边界，不重写原码流依赖

③ codec-video-prep readiness 组

候选帧上动态扩组



由重要性阈值、时间覆盖和边际收益共同结束

关键提醒 容器 stss 同步样本是随机访问入口，也不等于“视觉上最重要的一帧”

三者的关系可以压缩成一句话。编码 GOP 决定怎样解码，LLaVA 逻辑 GOP 决定在哪些时间段分配候选，readiness 组决定一批候选何时足够成熟并刷新画布。

2.7 为何有时规避关键帧

I 图片缺少帧间预测帮助，往往需要更多语法比特。若选择器直接按 bitcost 排名，I 图片可能凭编码结构获得过高分数。`avoid_keyframes=True` 会把命中同步样本的候选移到邻近非关键帧，再去重并补足数量。

这项策略没有普适最优值。完整 I 图片可提供稳定上下文，完全丢弃会损伤场景布局。更合理的做法通常是保留一个完整 anchor，同时避免让其他 I 区域凭码流成本重复占满稀疏预算。最终报告后文会看到，OneVision、LLaVA 与 Mage 都以某种形式保留完整 anchor 或 I patch。

第三章 从码流中能读出哪些视觉线索

压缩码流提供了哪些可用信号

信号越靠上越便宜、越偏时间；越靠下越细、越接近最终像素或模型语义



3.1 Packet size 只给时间强度

容器可以很便宜地告诉我们每个 packet 的字节数。把 P/B packet size 按时间桶累加, 可以找到编码成本突然上升的片段。快速运动、镜头切换、纹理激增或遮挡都可能形成峰值。

Packet size 没有直接空间坐标。它适合决定“何时多看”, 不适合单独决定“画面的哪一块值得保留”。LLaVA-OneVision-2 用它切自适应逻辑 GOP, JSONL codec-video-prep 链也用 packet energy 分配采样密度。

3.2 运动矢量提供块位移

运动矢量来自编码器的帧间预测决策。向量幅度大, 通常表示该预测块从较远位置取得参考; 向量变化剧烈, 可能对应局部运动边界。密集化时, 可以把一个 PU 或块的向量复制到其覆盖像素, 再聚合到 ViT patch。

它有几类常见误导。

- 摄像机平移会让大面积背景同时获得高运动。
- 静止但语义关键的物体可能获得零向量。
- 遮挡区的匹配不稳定，向量体现的是编码器妥协。
- 不同参考帧和四分之一像素插值会让幅度解释更复杂。

因此 OneVision-Encoder 将运动幅度和亮度残差结合，LLaVA-OneVision-2 继续加入 block bitcost。

Mage 对 HEVC 也使用加权运动与残差信号，并为神经 codec 改用概率码长。

3.3 残差体现预测失败程度

预测残差高，说明参考块加运动补偿后仍无法解释当前像素。新出现的物体、形变、细纹理和光照改变都会抬高残差。若只使用残差，传感器噪声、压缩噪声和水面纹理也可能长期占据预算。

Residual energy 常按亮度通道计算，再做尺度归一。它比运动幅度更能覆盖“物体没有大位移，但外观发生变化”的情况。两者结合后，选择器可以兼顾明显运动与预测困难。

3.4 Bitcost 综合编码难度

Bitcost 直接度量一段语法消耗的比特。它会同时吸收预测模式、运动信息、残差显著性和系数幅度的影响。codec-video-prep 的 HEVC 补丁在 CTU coding-quadtree 解码前后读取 CABAC 位置差，把可定位的 PU、TU 语法分给 4×4 子格，再把未分配余量摊回 CTU 区域。H.264 补丁在宏块 CABAC 或 CAVLC 路径做类似记录。VP9 使用 64×64 superblock 粗网格和子网格。

原生扩展在 AVFrame.opaque_ref 中读取一个带 magic、版本、尺寸和 stride 的缓冲区。粗粒度数组采用 int32，4×4 子网格采用 float32。复制到 NumPy 自有内存后，原始 AVFrame 可以安全释放。

【源码依据】native/cv_reader_fast.cpp 的 bitcost 结构与读取函数；ffmpeg_patch/bitcost_only/hevcdec.c 的 hevc_store_ctu_bit_cost、hevc_subcost_add_rect 与 CTU 解码位置记录；H.264 补丁中的 ff_h264_decode_mb_cabac 及对应 CAVLC 路径。

3.5 从编码网格到视觉 patch

编码块大小可变，ViT patch 固定。工程链需要完成三次坐标转换。

1. 把 CTU、MB、SB 或 4×4 子格成本展开成规则分数图。

2. 把分数图按准备后图像尺寸映射到像素坐标。
3. 对每个 14×14 或 16×16 patch 求和或平均，再把相邻 2×2 patch 聚成选择 block。

codec-video-prep 的 `bitcost_item_to_score_map` 会选择当前 codec 可用的粗细网格，可选 `log1p` 和分位数归一，然后用最近邻插值到像素图。最近邻不会凭空平滑边界，代价是块状感会保留下来。

【源码依据】`codec_selector/codec_patch_gop/video_processor.py` 中的 `bitcost_item_to_score_map`；`codec_selector/plugins/scorers/bitcost.py` 中的 `bitcost_items_to_score_maps`。

3.6 Codec 分数不等于任务注意力

压缩器优化的是重建质量和码率。用户可能问“桌上红色杯子旁边有什么”，杯子若一直静止，codec 分数会很低。另一个区域可能有水波和树叶，长期消耗大量比特，却与问题无关。

Codec-native 方法的价值来自去除普遍重复，而不是取代语义理解。视觉编码器和语言模型仍要学习任务相关性。安全的系统还需要回退机制，例如保留低分辨率全局视图、完整 anchor、少量均匀帧，或让问题条件影响第二阶段选择。

第四章 从完整帧到稀疏视觉 token

4.1 密集抽帧的计算瓶颈

假设每帧准备为 336×336 ，ViT patch 为 14。单帧有 24×24 ，也就是 576 个 patch。均匀抽取 64 帧会产生 36,864 个视觉 patch。若视觉编码器内部对全序列做标准自注意力，注意力矩阵规模随 token 数平方增长。许多系统会先逐帧编码或做局部注意力，但视觉主干、投影器和 LLM 上下文仍会被重复背景拖慢。

OneVision-Encoder 的默认论文示例保留 64 帧、GOP 32、总预算 2,048 token。若每个 GOP 分配一张完整 I 画布和若干稀疏 P patch，视觉 token 相比 64 张完整画面减少 87.5%。论文 Figure 5 显示被选 P patch 集中在运动和变化区域。

【论文依据】OneVision-Encoder Section 2.1、Figure 5 与 Appendix Table 8。

4.2 稀疏 token 带来的坐标问题

把来自不同帧的 patch 拼进一张 RGB canvas，可以直接复用普通图像处理器。画布坐标只表示“它被放在输出图片的哪里”，不再等于“它在原视频的什么时候、什么位置”。若模型只使用画布二维位置编码，两个完全不同时间和来源位置的 patch 可能被误认为相邻。

Codec-native 编码器因此需要来源记录。最小记录可以写成

```
u_i = (canvas_id, frame_id, src_h, src_w, packed_h, packed_w, group_id)
```

不同项目保留的字段数量不完全相同。codec-video-prep 的实际 src_patch_position.npy 是 $N \times 3$ 数组 [frame_id, patch_h, patch_w]，canvas 切分可以从输出图数量和 patch 容量推导。LLaVA-OneVision-2 论文使用更完整的 token record 记号。两者的抽象目的相同，存储格式不能直接画等号。

4.3 2×2 block 与视觉 merge 对齐

许多多模态视觉处理器会把相邻 2×2 patch 合并成一个下游视觉 token。若选择器逐 patch 取 Top-K，可能只保留一个 merge 单元里的部分 patch，导致形状不完整或需要复杂掩码。LLaVA-OneVision-2 与 codec-video-prep 都采用 2×2 patch block 作为基本选择单元，让打包顺序与 merge 结构一致。

设完整画面有 S_{full} 个 patch，组预算允许 M 张完整画布，block 边长为 b 。先保留一个完整 anchor，剩余 block 容量为

$$K = \text{floor}((M \times S_{full} - S_{full}) / b^2)$$

默认 $b=2$ ，每个候选 block 消耗四个 patch。所有非 anchor 帧的 block 可以跨时间全局竞争 K 个名额，也可以按时间桶预留配额。

4.4 Anchor 的作用

完整 anchor 提供场景布局、静止物体和低 codec 分数区域。稀疏 P patch 则补充变化。这个组合相当于一张低频稳定底图加若干时空更新，和预测编码的思想相呼应。

Anchor 不一定等同于码流 I 图片。公开 readiness 链常把逻辑组首个候选当 anchor；JSONL 链支持 sampled、keyframe 和 hybrid 选择；论文可能以固定 GOP 的 I 图片作为 anchor。阅读实现时必须查看 anchor 如何真正选出。

4.5 统一视觉编码器需要学会不规则输入

一个只在完整规则网格上预训练的 ViT，未必能自然理解打散 patch。OneVision-Encoder 从头训练视觉骨干，同时混合 codec 视频、普通分块视频和单图空间切片。3D RoPE 把时间、高度、宽度坐标注入注意力，使同一主干能处理规则和稀疏布局。

后文将继续区分两件事。论文提出了怎样的视觉编码原则，官方仓库又如何生成 token、坐标和注意力掩码。两者共同决定系统能否从稀疏画布恢复原视频结构。

第五章 OneVision-Encoder 把 codec 稀疏性变成视觉原语

5.1 贡献的准确边界

OneVision-Encoder 没有让 ViT 直接读取运动矢量，也没有用压缩系数替代像素。进入视觉编码器的内容仍是解码后的 RGB patch。运动矢量和预测残差只负责生成选择掩码，回答有限预算应花在哪些源 patch 上。

它真正建立了三个接口。

- 选择接口从压缩视频提取 MV 与 residual，输出值得保留的源 patch 坐标。
- 打包接口把稀疏 RGB patch 紧凑排列，使普通 Conv2d、FlashAttention 和批处理仍可工作。
- 位置接口把每个 patch 原来的 (t, h, w) 交给 3D RoPE，使物理存放位置和语义时空位置分离。

论文将这套原则称为 codec-aligned sparsity。它的重点是视觉 token 分配，而非压缩率优化。

【论文依据】OneVision-Encoder Section 1、2.1、2.2，Figure 1、2。

5.2 论文里的 HEVC 引导选择

论文 Section 2.1 给出一条容易理解的路径。

1. 视频按 GOP 组织，每个 GOP 含 I 图片和 P 图片。
2. 编码块的运动矢量广播到像素网格，取向量模长。
3. 解码亮度预测残差，计算局部能量。
4. 两类分数在 ViT patch 内聚合。
5. I 图片的 RGB patch 完整保留，P 图片保留高分 patch。

若一张完整画面有 P_0 个 patch，每个 P 图片保留比例为 r ，论文方法段写出的集合大小是 $\text{floor}(rP_0)$ 。不过后续实验设置和官方代码使用整段 clip 全局 Top-K 固定预算。这个差异会影响时间覆盖，后文单独解释。

5.3 三种输入形态共用一个视觉骨干

论文 Section 2.2 把训练输入分为三类。

Dense Video-Codec Patchification

Codec 视频保留完整 I patch，并从 P 图片选择稀疏 RGB patch。默认示例含 64 帧、 224^2 、patch 14。每帧有 16×16 ，也就是 256 个 patch。密集候选总数为 16,384，最终预算 2,048，只保留 12.5%。

Chunk-wise Patchification

普通视频不依赖 codec 信号。时间轴被分成若干 chunk，每个 chunk 随机选一帧，保留整帧规则网格。它给模型提供均匀时间覆盖，也让视觉骨干不会只适应 codec 稀疏布局。

Single-image Spatial Patchification

单图按规则二维网格切 patch，时间坐标设为零。高分辨率图像还可用全局缩略图和空间 tile 组合。

三类输入共享 ViT、3D RoPE 和 attentive pooling。论文 Appendix Table 9 给出一个批内比例 50%、37.5%、12.5，对应 codec、chunk 和 collage 或 image 形态。公开训练 shell 中只有特定 large 配置恰好落实这一比例，不能把它当作所有训练任务的硬编码默认值。

OneVision-Encoder 的统一输入思想

同一视觉骨干接收 codec 稀疏视频、分块抽帧视频和单图空间切片



来源依据 OneVision-Encoder § 2.1、§ 2.2、§ 2.5, Figure 4, Appendix § 8

5.4 模型本体怎样处理视频

审计仓库 HEAD 为 c2cd3e0d85c7c44d1b1b1bad5317ceb6ff6fe93a。论文发布日前最后提交为 099d44d10e59382695428a127a1dd959fdf00b04。二者之间核心模型、训练、codec 与 LLaVA 源码没有变化，后续提交主要更新说明文档。

Large 配置使用 24 层、hidden size 1024、16 个 attention heads、MLP 4096、patch 14。每个 head 的维度是 64。OneVisionEncoderEmbeddings 先把 5D 视频 [B,C,T,H,W] 变成 [B×T,C,H,W]，再用 Conv2d(kernel=stride=14) 生成二维 patch，随后把时间重新折回 token 序列。这里没有 tubelet 和 3D 卷积，时间关系由 token 顺序与 3D RoPE 注入。

Transformer 使用双向视觉注意力。FlashAttention2 调用明确设置 `causal=False`。末端 Siglip2MultiheadAttentionPoolingHead 用一个可学习 probe 对全部 token 做多头注意力，再经 RMSNorm 和 MLP 得到整段表示。

【论文依据】Appendix Section 8、Table 7。

【源码依据】onevision_encoder/configuration_onevision_encoder.py 的

OneVisionEncoderConfig；onevision_encoder/modeling_onevision_encoder.py 的 OneVisionEncoderEmbeddings、

OneVisionEncoderFlashAttention2.forward、Siglip2MultiheadAttentionPoolingHead。

5.5 3D RoPE 怎样恢复原位置

标准二维 ViT 位置只描述画面高宽。OneVision 为每个 token 准备时间、高度、宽度三个整数坐标。VideoRotaryEmbeddingSplit466 按 $T:H:W = 4:6:6$ 分配 rotary 频率。Large 每个 head 有 64 维，内部先产生 32 个 half dimensions，其中时间占 8，高度和宽度各占 12，随后复制到 Q/K 所需的 64 维。

两个 token 的旋转相位差编码 $(\Delta t, \Delta h, \Delta w)$ 。Codec patch 即使被搬到另一张 canvas，只要位置表仍写源帧和源网格，注意力便能感知它原来的相对时空关系。

代码支持两种位置入口。

- `visible_indices` 是源 64 帧规则网格的一维索引。模型可从虚拟完整网格查出 3D 频率。
- `patch_positions` 直接提供 $[B, L, 3]$ 的显式坐标，适合原生分辨率、非方形画面和稀疏 canvas。

【论文依据】Section 2.5、Figure 4。

【源码依据】`onevision_encoder/modeling_onevision_encoder.py` 中的 `VideoRotaryEmbeddingSplit466`、`OneVisionEncoderModel.forward`。

5.6 `visible_indices` 的两个角色

`visible_indices` 在不同入口里有两种语义，名字相同却不能混用。

若模型收到完整 64 帧 patch embedding，且可见数量小于总 patch 数，

`OneVisionEncoderModel.forward` 会按索引真正 gather hidden states。

官方预训练主链更早就在数据侧 gather 了 2,048 个 RGB patch，并把它们重排成 8 张 224^2 伪帧。模型 embedding 数量已经是 2,048，与 `visible_indices` 数量相等，前向不再 gather。此时索引只从虚拟 64 帧 RoPE 网格中查源位置。

这个技巧保留了规则张量和成熟 GPU kernel，也带来严格对齐要求。RGB patch 顺序与索引只要错一位，张量形状仍完全合法，模型却会给内容配上错误时间和空间坐标。

5.7 论文中的监督目标

OneVision-Encoder 选择用离线聚类标签从头训练视觉骨干。冻结的 MetaCLIP-H/14 先提取图像和视频特征，再聚成大规模视觉中心。图像使用约 200 万中心，视频使用约 40 万中心。OCR 数据额外从识别文本生成词标签。

论文 Section 2.4 的 Equation 9 把目标写成多标签 logistic 形式。公开代码实际使用 `PartialFC_V2`。一条样本虽存 Top-10 标签，训练时随机取其中 8 个，分别计算带 margin 的 sampled softmax 交叉熵，再合并损失。这是一项实质实现差异。

论文对视频聚类帧数也有内部不一致。Section 2.3 写 16 帧，Section 3 写 8 帧，工具默认 8 帧。

【论文依据】Section 2.3、2.4、3，Equation 9、Table 1。

【源码依据】tools/kmeans 下的特征与聚类脚本；training/fused_partial_fc_v2_multi_res.py 中的 `PartialFC_V2`。

5.8 离线训练资产怎样生成

公开工具展示的预训练资产链如下。

```
原视频
→ step1_offline_preprocess.py
    方形化、64 帧、224²，先写 H.264 MP4
→ step2_convert_videos2hevc.py
    libx265 重编码、无 B 图片
→ step3_generate_video_*_index.py
    生成每条视频的 .visidx.npy
→ 训练清单
    video_path + 10 labels + visidx_path
```

`step2_convert_videos2hevc.py` 默认 `GOP_SIZE=16`、关闭 `scenecut`、`bframes=0`、`ref=1`、`CRF 23`。

论文 Stage 2 写的是 `GOP 32`、每 64 帧两张完整 I 图片。公开脚本默认值与论文不一致，复现者需要显式调整。

仓库有 `residual-only` 与 `MV+residual` 两个 `step3`。

Residual-only 脚本把 I 图片 residual map 先设为数值 0，随后统一执行 `abs(residual-128)`。这会让 I 区域得到 128 的高能量，常被全局 Top-K 优先选中。它没有显式预留两张 I 图片。脚本默认 patch 16、K 2000，也需改为 patch 14、K 2048 才对齐论文示例。

MV+residual 脚本实现了每帧 95 百分位归一和等权融合，然而审计提交有确定性缺陷。

`process_one_video_mv_res` 在局部变量 `T` 赋值前执行 `Tsel=T`，异常被外层捕获后返回全零索引。

即使交换两行，`frames_collected` 也没有递增，后续会用最后一张能量图覆盖时间序列。公开脚本无法原样证明发布权重的有效 MV+res 资产怎样生成。

`step1_offline_preprocess.py` 还导入仓库中缺失的 `dataloader.data_decord_video`。因此这条离线链需要补写 loader 或取得未公开依赖，无法从当前仓库一键复现。

【源码依据】 `tools/tools_for_hevc/step1_offline_preprocess.py`、`step2_convert_videos2hevc.py`、`step3_generate_video_residual_index.py`、`step3_generate_video_mv_residual_index.py`。

5.9 DALI loader 的真实记录和对齐风险

`data_decord_codec.py::VideoExternalSource` 读取每行四部分信息。

```
video_path
10 个 video cluster labels
.visidx.npy path
```

Loader 用 Decord 取得 64 帧，DALI resize 到 224^2 并归一化，输出 `[B,3,64,224,224]`。索引以 int16 加载，训练代码取前 2,048 个。

代码还有一个值得复现者立即修复的边界风险。恰好 64 帧且 `sequence_length=64` 时，`_get_frame_indices` 的直接 `range(64)` 条件写成 `num_frames < sequence_length`，等于时仍走随机分段取整。结果可能重复或跳过帧，也取不到最后一帧。离线 `.visidx` 常按转码视频槽位 0 至 63 生成，loader 没有载入生成时的 frame-id 表，也未重映射。公开链因此存在 RGB 槽位和 visidx 时间坐标错配风险。

这项风险只能证明公开代码可能错配，不能倒推出已发布权重必然受影响。最终训练可能使用了不同资产或私有 loader，仓库没有提供足够证据。

【源码依据】 dataloader/data_decord_codec.py 中的 _get_frame_indices 与 VideoExternalSource。

5.10 训练主循环的张量链

入口 training/train.py::main 的 codec 视频路径可以按形状追踪。

源 RGB	[B, 3, 64, 224, 224]
完整 patch	[B, 3, 16384, 14, 14]
visible_indices	[B, 2048]
gather 后 patch	[B, 3, 2048, 14, 14]
规则伪视频	[B, 3, 8, 224, 224]
Conv patch embedding	[B, 2048, 1024]
24 层输出	[B, 2048, 1024]
attentive pooling	[B, 1024]
PartialFC 分类损失	标量

训练批次还混入 chunk 样本。代码从 8 个时间 bin 各随机抽一帧，构造 2,048 个完整帧 patch index，再与 codec 样本一起 gather 和重排。Collage 分支把 8 张 224² 图纵向拼成一张 [B, 3, 1792, 224] 的 2D 图，时间坐标退化为零。

论文 chunk-wise 3D RoPE 用 chunk id。代码保留随机抽中的真实 0 至 63 frame id。这是合理的实现修改，能保留更精细时间差，但与论文公式并不逐项相同。

【源码依据】 training/train.py::main 的视频分

支； onevision_encoder/modeling_onevision_encoder.py::OneVisionEncoderModel.forward。

5.11 仓库内置 LLaVA 路径

OneVision 仓库已经包含一条连接 Qwen3 的 LLaVA-NeXT codec 原型，这条链为后续工作提供了清晰桥梁。

Stage 1 在原视频上均匀取 64 帧，把画面缩放并补成 576²，以 patch 16 得到 36×36 选择网格。它融合 MV 与 residual，整段 global Top-K。训练 demo 打开 keep_first_full_frame，强制第一张 sampled frame 的 1,296 个 patch 全保留，其余预算从后续帧竞争。

Stage 2 把选中 patch 紧密填入 8 张 576² canvas，另存 positions_thw.npy。被打包位置只决定 RGB 在 canvas 哪里，位置表继续保存源 (t,h,w)。mm_utils.process_images 把 576² resize 成 504²。选

择网格 patch 16 和 encoder patch 14 都得到每边 36 格，于是一个 selector cell 精确对应一个视觉 patch。

8 × 36 × 36 = 10,368 个视觉 patch

数据集加载 positions_thw 后，collator 传给 prepare_inputs_labels_for_multimodal。
OneVisionEncoderVisionTower.forward 把 [8,3,504,504] 变成 [1,3,8,504,504]，传显式位置到 OneVision encoder，再经两层 MLP projector 接入 Qwen3。

视觉塔 wrapper 对位置 list 直接取 [0]，官方 SFT 每设备 batch 为 1 时可用。若把异构 codec 样本 batch 提升到 2 以上，它会只使用第一个样本位置表，需要改成逐样本或真正 batch 化。

【源码依据】llava_next/Compressed_Video_Reader/tool/stage1.py、stage2.py；llava_next/llava/mm_utils.py::process_images；llava_next/llava/model/multimodal_encoder/onevision_encoder.py::OneVisionEncoderVisionTower.forward；llava_next/llava/model/llava_arch.py。

5.12 论文与源码的关键差异

议题	论文表述	审计代码	结论
P patch 预算	Section 2.1 逐 P 图片固定比例	训练、probe 和 LLaVA 原型都用整段 global Top-K	实验实现改为全局固定预算
GOP	Stage 2 使用 GOP 32	转码工具默认 GOP 16	默认脚本不对齐论文
I patch	64 帧含两张 I，全部保留	不同脚本分别高分、清零或只强制首 sampled frame	公开实现没有统一规则
MV+res	论文核心选择信号	有代码，但预训练 step3 存在确定性 bug	无法原样复现有效资产
训练目标	多标签 logistic	8 路 margin-softmax PartialFC	损失实现不同
Chunk 时间	chunk id	实际随机 frame id	代码保留更细时间位置
视频聚类帧数	Section 2.3 写 16，Section 3 写 8	工具默认 8	论文内部不一致
Stage 2 规模	lr 1e-4、4B samples	公开 large shell 为 lr 5e-5、320M 采样量	公开示例不足以复现论文完整作业

5.13 该怎样理解 OneVision-Encoder

OneVision-Encoder 最稳固的贡献是“稀疏 RGB 内容、源位置 3D RoPE、规则计算布局”这组三件套。具体 MV/res 预处理脚本存在缺口，论文与代码也有多个口径差异，但模型接口把 codec 选择器从视觉骨干中解耦出来。后续工作可以更换 bitcost、神经 codec 码长或任务条件选择器，而无需重新定义 RGB patch 的内容。

这种接口分离直接通向两个方向。`codec-video-prep` 把选择与打包做成跨 codec 预处理基础设施。LLaVA-OneVision-2 把固定 clip global Top-K 改成自适应时间分组和 2×2 block 预算。Mage-ViT 则从头训练支持可变长度与 padding mask 的 codec-native 视觉编码器，并把它接到流式多模态系统。

第六章 codec-video-prep 如何把码流变成模型输入

6.1 项目定位与审计版本

codec-video-prep 是这条研究路线里最靠近系统工程的一层。它不训练视觉模型，主要负责把普通视频文件转换为少量 RGB canvas 和来源位置数组。下游仍可调用现有 image processor，无需把 patched FFmpeg 逻辑塞进模型前向传播。

本文审计的仓库提交为 77e8e91c11bb2fd520701e49465c9001f6c5b8ad。pyproject.toml 声明包版本 0.2.5，该提交位于 0.2.5 标签之后，包含 2026 年 6 月 3 日的主分支修订。复现时应同时记录包版本和提交哈希。

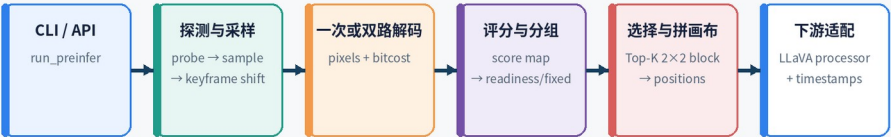
仓库包含几层代码。

层	主要目录	职责
公开包	src/codec_video_prep	CLI、配置、单视频 API、LLaVA 适配
选择器核心	codec_selector/core	探测、解码、主流水线、统一配置
可插拔策略	codec_selector/plugins	采样、评分、分组、Top-K 选择
数据集链	codec_selector/codec_patch_gop	JSONL 批处理、packet energy、mvres 与资产写入
原生扩展	native	从 patched FFmpeg 的 AVFrame 读取 bitcost 与像素
FFmpeg 补丁	ffmpeg_patch	H.264、HEVC、VP9 解码器内部成本采集

codec-video-prep 的真实调用链

仓库保留两条流水线；公开单视频链与 JSONL 数据集链的默认策略不同

公开单视频 readiness 链



JSONL 数据集 GOP 链



审计提交 77e8e91 包声明 0.2.5，提交晚于 0.2.5 标签

6.2 仓库有两条流水线

公开 CLI codec-video-prep 与 Python API run_preinfer() 使用单视频 readiness 链。

```
codec_video_prep.cli.main
→ codec_video_prep.api.run_preinfer
→ codec_video_prep.api.run_preinfer_config
→ codec_selector.core.pipeline.run_bitcost_readiness
```

JSONL 数据集入口使用另一条链。

```
python -m codec_selector.codec_patch_gop.main
→ multiprocessing.Pool
→ codec_selector.codec_patch_gop.video_processor.process_one_video
```

两条链共享 patched FFmpeg 和若干采样、几何、打包工具。它们的默认评分源、组的定义、锚帧策略、RGB 解码时机和输出资产并不相同。把第二条链的论文复现策略当作公开 run_preinfer() 的默认行为，会得到错误结论。

比较项	公开单视频链	JSONL 数据集链
默认评分	bitcost	mvres
时间采样	uniform_count 等公开模式	packet energy 驱动候选

比较项	公开单视频链	JSONL 数据集链
分组	readiness、fixed 或内部 adaptive_gop	packet energy 的可变逻辑 GOP
锚帧	通常是逻辑组首帧	sampled、keyframe、hybrid
画布预算	images_per_group	每桶一张完整 anchor 加固定数量 P 画布
批量并行	由调用方管理	内置视频级进程池

6.3 公开配置怎样进入核心

src/codec_video_prep/config.py 定义 PreinferConfig。run_preinfer() 提供同名关键字参数，默认包含 1024 个候选帧、uniform_count、group_size=32、images_per_group=4、patch_size=14、block_size=2、max_pixels=153664、avoid_keyframes=True 和 decode_backend=cv_reader_pixels。

run_preinfer_config() 把公开对象映射成内部 BitcostReadinessConfig。这一层还固定了若干策略。

内部字段	固定值	作用
bpppf_clamp_min	0.015	码率密度归一的下界
bpppf_clamp_max	0.09	码率密度归一的上界
readiness_coverage_bins	3	readiness 需要覆盖的时间桶数
readiness_delta_ratio	0.05	新帧边际收益的停止阈值
frame_score_norm_mode	none	公开入口默认不做逐帧均值归一

api._configure_native_threads 会在加载原生解码工作前设置 CV_READER_FAST_THREAD_TYPE、CV_READER_FAST_THREAD_COUNT 与 CVR_DISABLE_TARGET_ONLY。这些变量影响 FFmpeg 内部线程和目标帧过滤。它们与 Python 线程池、分段进程池、视频级进程池属于不同并行层次。

【源码依据】src/codec_video_prep/api.py 中的 run_preinfer、run_preinfer_config、_configure_native_threads；src/codec_video_prep/config.py 中的 PreinferConfig。

6.4 第一步 探测视频元数据

`codec_selector.core.probe.probe_video` 调用 `ffprobe`，读取第一条视频流和容器信息，整理出总帧数、fps、高宽、编码器名、流级码率、容器码率和时长。主链会检查帧数与尺寸是否为正，随后所有 `frame id`、数组长度和网格尺寸都依赖这些值。

码率用于 `readiness` 阈值的尺度校准。码率缺失时会从其他字段回退。这里得到的 `fps` 是后续把 `frame id` 换成秒数的基础。可变帧率视频更适合保留真实 PTS，单纯用 `frame_id / fps` 只是近似。

6.5 第二步 候选帧采样

`codec_selector.plugins.samplers.basic.sample_frame_ids` 接收采样模式。

- `uniform_count` 在完整帧号区间用 `numpy.linspace` 生成固定数量候选。
- `all_frames` 返回全部 `frame id`。
- `pkt_size_peak` 选择 `packet size` 峰值附近的帧。
- `fps_plus_pkt_size_peak` 合并固定时间密度和峰值候选。
- 其余字符串进入 `fps` 采样分支。

公开 CLI 暴露的枚举含 `pkt_peak`，内部插件识别的名称却是 `pkt_size_peak`。在审计提交上，用户输入 `--frame_sampling_mode pkt_peak` 会静默进入 `fps` 分支，并不会执行包峰值采样。这是代码级不匹配，不能按 README 的意图推断运行结果。

【源码依据】`src/codec_video_prep/cli.py` 的参数 `choices`；`codec_selector/plugins/samplers/basic.py` 的 `sample_frame_ids`。

6.6 第三步 规避同步样本

启用 `avoid_keyframes` 后，MP4 优先由 `mp4_keyframe_frame_ids` 解析

`moov/trak/mdia/minf/stbl/stss`。 `stss` 使用从 1 开始的 `sample number`，代码转成从 0 开始的 `frame id`，并在需要时补入 0。非 MP4 或解析失败时，`ffprobe_keyframe_frame_ids` 使用 `-skip_frame nokey` 取得关键图片时间戳，再映射到帧号。

命中的候选会移动到附近非关键帧。平移后可能重复，主链继续去重，从尚未使用的非关键帧补足候选数量。`all_frames` 没有额外帧可补，因而忽略规避请求。

这项处理规避的是解码随机访问点偏高的编码成本，并不在做镜头关键画面检测。

【源码依据】`codec_selector/codec_patch_gop/video_probe.py` 中的 `mp4_keyframe_frame_ids`、`ffprobe_keyframe_frame_ids`；`codec_selector/core/pipeline.py` 中的规避与补齐分支。

6.7 第四步 准备几何尺寸

`resolve_prepared_frame_geometry` 先按照 `max_dim` 与 `max_pixels` 等比例缩放，再只在右侧和下侧补零。最终高宽必须同时被 `patch_size × block_size` 整除。默认 `patch 14`、`block 2`，因而高宽要对齐到 28 的倍数。

只在右下补边可以保持原图左上角为坐标原点。来源位置 `[frame_id, patch_h, patch_w]` 无需为左上 padding 再做平移。`prepare_frames` 会把所有帧统一到这个几何尺寸，并检查数组形状。

【源码依据】`codec_selector/core/frame_ops.py` 中的 `resolve_prepared_frame_geometry` 与 `prepare_frames`。

6.8 第五步 取得 RGB 和 bitcost

公开默认后端 `cv_reader_pixels` 走单次扫描。`patched FFmpeg` 解码器把 bitcost map 附在 `AVFrame.opaque_ref`，C++ 扩展同时用 `swscale` 把重建帧转为目标尺寸的 BGR NumPy 数组。像素和成本来自同一个 `AVFrame`，天然按 `frame id` 对齐。

兼容后端 `ffmpeg_native` 分成两项。`FFmpeg` 子进程通过 `select filter` 解码目标帧 BGR，`cv_reader_fast` 单独扫描 bitcost。`parallel_decode_cv_reader=True` 时，主链用两个 Python 线程同时启动外部 `FFmpeg` 和 C++ 工作。Python GIL 不是主要瓶颈，重计算发生在子进程和原生代码中。

特性	<code>cv_reader_pixels</code>	<code>ffmpeg_native</code>
视频扫描	一次	像素与成本分两路
对齐	同一 <code>AVFrame</code>	按 <code>frame id</code> 合并
Python 双任务并发	不需要	可选
典型用途	默认生产路径	兼容、诊断、独立解码验证

`codec_selector.core.decode.decode_selected_frames_ffmpeg` 会按唯一 frame id 批量解码。若个别目标帧丢失，代码使用最近或最近已成功帧回填。`cv_reader_fetch_bitcost` 也有串行回退与缺失项回填。回填可维持固定数组形状，却会在时间上复制邻帧。元数据和质量评估应统计这类事件。

【源码依据】`codec_selector/core/pipeline.py` 的解码后端分

支；`codec_selector/core/decode.py`；`codec_selector/codec_patch_gop/video_processor.py` 中的 `cv_reader_fetch_bitcost`。

6.9 原生扩展和 patched FFmpeg

`native/cv_reader_fast.cpp` 为三类编码定义了不同粗网格。

编码	粗网格	细网格
H.264	16×16 宏块 MB	4×4 子块
HEVC	CTU	4×4 子块
VP9	64×64 superblock	4×4 子块

每个 bitcost 缓冲区有固定头部、粗粒度 `int32` 数组和细粒度 `float32` 数组。C++ 先校验 magic、版本、宽高和 stride，再复制到 NumPy。`read_video_fast_selected` 总是允许跳过 loop filter；只有不导出像素时才允许 `skip_idct`。当默认路径同时输出 RGB 时，逆变换仍要执行，得到可用重建画面。

HEVC 补丁在 CTU coding-quadtrees 入口和出口记录 CABAC bit position。`hevc_store_ctu_bit_cost` 保存总成本，`hevc_subcost_add_rect` 把可定位语法成本分给覆盖的 4×4 单元，CTU 余量再均摊。H.264 CABAC 路径在 `ff_h264_decode_mb_cabac` 前后记录宏块成本，并对细语法做区域分配。这个方案取得了真实熵码流位置差，空间归因仍是一种守恒式近似。

主链可以把 selected frame id 分成若干带 guard frames 的区段，交给 `ProcessPoolExecutor` 并行扫描。每个 worker 内部还可使用 FFmpeg slice 或 frame threading。它们分别是跨区段进程并行和单解码器内部并行。

6.10 第六步 分数图与坏帧掩码

像素准备完成后，主链可并行检测异常或坏帧。bitcost item 经 `bitcost_items_to_score_maps` 进入 `bitcost_item_to_score_map`。后者根据 codec 选择 MB、CTU、SB 或子网格，先截断负值，可选 `log1p`，再按分位数归一到 0 至 1，最后最近邻放大到像素图。

主链可裁剪 I 图片分数，也支持逐帧归一。公开映射把 `frame_score_norm_mode` 固定为 `none`。之后一次性预计算每个 patch 和 block 的分数，避免 readiness 扩组时反复聚合像素图。

6.11 第七步 三种分组策略

fixed

固定策略按给定帧数直接切候选列表。它可预测、易于批处理，但不能因内容变化调整时间范围。

adaptive_gop

公开核心里保留 `adaptive_gop` 分支，依据帧级总代价切逻辑组。它与码流真实 GOP 仍是两回事，也与 JSONL 链的 `packet energy` 算法不同。

readiness

`readiness` 先估计一组在给定画布预算下可选多少 block。设完整帧 patch 数为 `S_full`，每组画布预算为 `M`，block 边长为 `b`。一张完整 anchor 消耗 `S_full`，剩余 block 容量为

$$K = \text{floor}((M \times S_{\text{full}} - S_{\text{full}}) / b^2)$$

`compute_readiness_stats` 排除 anchor 后，把当前组所有候选 block 全局排序，累加前 `K` 个分数，并统计它们覆盖的时间桶。`build_readiness_groups` 从最小组长开始逐帧扩展。累计重要性达到阈值、时间覆盖足够，并满足边际增益停止条件时可以刷新；达到最大组长也会强制刷新，尾部由专门逻辑收束。

Readiness 组长由 `min_group_frames` 与 `max_group_frames` 直接限制。`group_size` 参与默认阈值估计和预算基准，README 将它解释成最大组长会造成误导。

【源码依据】`codec_selector/plugins/groupers/readiness.py` 中的 `_capacity_blocks`、`compute_readiness_stats`、`estimate_readiness_threshold`、`build_readiness_groups`。

6.12 第八步 跨帧 Top-K 选择

每个组进入 `process_group_topk_2x2`。

1. 选定一张 anchor，保留其完整 patch 网格。
2. 排除坏帧和 anchor，对其他帧的 2x2 block 计算分数。
3. 所有候选可跨帧全局排序，也可结合时间桶做事件聚合。
4. 选取预算允许的前 K 个 block。
5. 排序时先放 anchor block，其余按来源帧和空间位置稳定排列。
6. 数量不足时补零到固定容量。
7. 交给 `pack_patches_to_canvases` 写入 RGB canvas。

跨帧全局 Top-K 会把预算集中到强事件，也可能让少数帧垄断名额。时间桶覆盖和分层配额正是为缓解这一问题。LLaVA-OneVision-2 的论文进一步显式设计了分层时间 P 画布分配。

【源码依据】`codec_selector/plugins/selectors/topk_2x2_bitcost.py` 中的 `process_group_topk_2x2`。

6.13 第九步 画布打包与来源坐标

`pack_patches_to_canvases` 按 block-raster 顺序把 patch 写入输出画布。源帧是 OpenCV 风格 BGR，写图前转为 RGB。函数同时返回 N×3 位置数组 `[img_idx, patch_h, patch_w]`。主链再把组内局部 `img_idx` 映射回全局来源 frame id。



README 对 `src_patch_position.npy` 的字段描述与代码不一致。审计提交中，选择器、保存逻辑和 LLaVA 适配都把它当作 $N \times 3$ `[frame_id, patch_h, patch_w]`，并非每行含边界框的六维或更高维结构。调用方应以数组 `shape` 和函数实现为准。

【源码依据】`codec_selector/codec_patch_gop/patch_utils.py` 中的 `pack_patches_to_canvases`；`codec_selector/core/pipeline.py` 中的位置回填和保存。

6.14 输出资产

公开链的 `PreinferResult` 返回 `out_dir`、`meta_path`、`canvas_files` 和 `summary`。位置数组与 `frame id` 作为输出目录中的资产读取。典型文件如下。

资产	含义	使用注意
canvas_*.png	模型可直接读取的 RGB 拼接图	内部可能含零 padding
src_patch_position.npy	每个输出 patch 的原 frame id 与网格坐标	实际 shape 为 $N \times 3$
frame id 数组	候选与最终来源的时间索引	应结合 fps 或 PTS 解释
meta.json	配置、几何、分组、性能等摘要	部分字段存在实现缺陷

审计提交中，`run_bitcost_readiness` 写摘要时把 `decode_backend` 硬编码为 `ffmpeg_native`。默认运行即使真实走 `cv_reader_pixels`，该字段仍可能显示错误后端。验证性能时需要同时保存调用配置和运行日志。

【源码依据】`codec_selector/core/pipeline.py` 中 `summary` 的构造；`src/codec_video_prep/api.py` 中的 `PreinferResult`。

6.15 JSONL 数据集链的真实流程

`codec_selector.codec_patch_gop.main` 读取 JSONL，过滤输入后用 `multiprocessing.Pool` 把不同视频分给 worker。每个 worker 调用 `process_one_video`。这层视频级并行与公开链里的解码线程、分段进程并行可以叠加，配置过大时会造成 CPU 过度订阅。

数据集链默认评分源是 `mvres`，也支持 `bitcost`。`build_variable_length_gops_by_energy` 排除 I packet，把 P/B packet size 汇入时间桶。代码根据累计能量配额寻找组边界，受最小和最大长度约束，并在局部低谷附近微调。得到的是服务视觉采样的逻辑 GOP。

每个桶使用一张完整 anchor 画布和若干稀疏 P 画布。Anchor 可来自 `sampled frame`、码流 keyframe 或 hybrid 策略。`process_one_video` 还写入更多时间、位置和资产索引，适合离线构建训练数据。

这条链更接近 LLaVA-OneVision-2 论文的 adaptive logical GOP 和固定 I/P 画布配方。公开 `run_preinfer()` 的 `readiness` 链则更像一个易调用、模型无关的单视频推理接口。

【源码依据】`codec_selector/codec_patch_gop/main.py`；`codec_selector/codec_patch_gop/energy_sampling.py` 中的 `build_variable_length_gops_by_energy`；`codec_selector/codec_patch_gop/video_processor.py` 中的 `process_one_video`。

6.16 LLaVA 适配器怎样消费结果

`LlavaOneVisionCodecPreprocessor.preprocess` 先根据视频和配置生成缓存键，用 `fcntl` 文件锁避免多个进程重复构建。`load_llava_codec_result` 读取画布、 $N \times 3$ 位置和元数据。`prepare_llava_onevision_inputs` 把每张 canvas 当作普通图像交给 processor，取得 `pixel_values` 与 `image_grid_thw`。

下游 processor 往往先按 raster 切 patch，再做 2×2 merge。convert_positions_to_block_layout 把来源位置换成对应顺序，codec_positions_for_processor 按每张图的 image_grid_thw 切分。时间戳函数把 frame id 换成时间文本，并在 vision token 段之间插入时间提示。最终返回 input_ids、attention_mask、pixel_values、image_grid_thw 和 patch_positions。



适配器配置含 target_canvas。代码会校验它，并用它反推候选帧数量，随后 _run_locked 调用 run_preinfer 时没有把固定目标画布约束传到底层。因此该参数会影响候选规模，却不能保证最终 canvas 数严格相等。

【源码依据】src/codec_video_prep/integrations/llava_onevision.py 中的 LlavaOneVisionCodecPreprocessor、load_llava_codec_result、convert_positions_to_block_layout、codec_positions_for_processor、prepare_llava_onevision_inputs。

6.17 一次端到端数据流示例

假设输入视频有 10,000 帧，公开配置均匀取 1,024 个候选，准备后网格为 24×24 patch，每组预算四张画布，block 为 2×2。

1. 探测器得到 fps、尺寸、编码器与码率。
2. uniform_count 在完整时间轴生成 1,024 个 frame id。
3. 命中同步样本的候选被移动、去重并补齐。

4. 单次解码同时得到 1,024 张准备后 BGR 与对应 bitcost map。
5. 每张图有 576 个 patch，一张完整 anchor 消耗 576。
6. 四张画布总容量为 2,304 patch，剩余 1,728 patch 可容纳 432 个 2×2 block。
7. Readiness 扩展当前组，直到前 432 个候选 block 的累计分数、时间覆盖和边际条件满足。
8. 组内 anchor 的 576 个 patch 全部保留，其余帧共同竞争 432 个 block。
9. 选择结果转换为 RGB canvases 和来源位置，零填充补齐固定容量。
10. LLaVA 适配器按 processor 的 merge 顺序重排位置，并加入 frame timestamp。

示例数字用于解释预算关系。实际组数、画布数和选中帧分布由输入内容和 readiness 参数决定。

6.18 复现与调试清单

- 同时记录 Git 提交、包版本、patched FFmpeg 构建和系统 FFmpeg 路径。
- 保存实际传入配置，不只信任 meta.json 的 decode_backend。
- 对采样模式打印首尾 frame id，当前提交不要把 pkt_peak 当作已验证的 packet peak。
- 检查 src_patch_position.npy.shape[1]，按 N×3 解析。
- 统计缺失帧回填、坏帧、padding patch 和每组实际长度。
- 分别测量探测、解码、bitcost、分组、打包、processor 和模型前向时间。
- 控制视频级进程、分段进程和 FFmpeg 内部线程的乘积，避免过度并行。
- 用人工可视化把选中 block 叠回原视频，验证时间、空间和 2×2 merge 顺序。

6.19 源码审计结论

codec-video-prep 的技术价值在于把压缩器内部信号变成模型无关的标准资产。主链涵盖解码器补丁、网格投影、动态分组、跨帧预算、稳定打包和坐标恢复，工作范围远超运动矢量抽取。仓库内两条流水线和若干 README 差异说明，复现必须沿真实入口追踪，不能只按名词对齐论文。

第七章 LLaVA-OneVision-2 把稀疏视觉接进通用 MLLM

7.1 它解决的下一层问题

OneVision-Encoder 证明了稀疏 RGB patch 与源时空坐标可以由同一视觉骨干处理。LLaVA-OneVision-2 继续处理三个系统问题。

- 长视频内容强度不均匀，固定 GOP 和固定每帧比例会浪费预算。
- 稀疏 patch 要与 2×2 spatial merge、局部视觉注意力和语言占位 token 严格对齐。
- Codec 视频、普通抽帧与图像要进入同一 connector 和 Qwen3，不增加 codec 专用语言分支。

论文 Section 2.1 明确采用 OneVision-Encoder 作为共享 backbone。视觉特征经两层 MLP 投影到 Qwen3-8B 的 4096 维词嵌入空间，替换文本序列中的视觉占位符。Codec 只改变证据选择和视觉分组，LLM 仍做标准自回归训练与生成。

【论文依据】LLaVA-OneVision-2 Section 2.1、Figure 2。

7.2 论文中的 codec 前端

论文把前端写成 $C(V)=(X,U,G)$ 。

- X 是完整 anchor 或 I canvas 与稀疏 P canvases 的 RGB 图像。
- U 是 token 记录，包含 canvas 索引、源帧、打包坐标、源空间坐标和 group id。
- G 是自适应组的边界与组信息。

一张 P canvas 不对应视频里真实存在的一帧。它是一张由多个时刻的高价值 2×2 block 拼成的图。 U 负责恢复每个块原来的时间和空间位置。

【论文依据】Section 2.2、Figure 3、4。

7.3 论文的自适应逻辑 GOP

论文先忽略 I packet，把 P/B packet bitcost 汇入时间桶。设 P/B 总成本为 B_{total} ，期望得到 N_g 个组，则组配额近似为

$$\theta = B_{total} / N_g$$

从当前组起点向后累积 packet cost。达到最短组长后，首次超过配额会形成候选边界；达到最长组长则强制截断。算法还在候选附近寻找 bitcost 局部低谷，让边界更可能落在内容变化较弱的位置。

变化剧烈的片段单位时间消耗更多比特，较快达到配额，组会缩短。平稳片段累积较慢，组会延长。这是在固定总体预算下自适应分配时间分辨率。

它与 `codec-video-prep` 当前公开主链的 `readiness` 算法不同。`Readiness` 先均匀取得候选帧，再观察当前组前 K 个 block 的累计分数、时间覆盖和边际收益。公开代码另有名为 `adaptive_gop` 的选项，按单帧总代价分位数切分，也不等于论文的累计配额与局部谷值公式。

【论文依据】Section 2.2、Equation 2 至 4、Figure 3。

【源码依据】`codec_selector/plugins/groupers/readiness.py`；`codec_selector/core/pipeline.py` 中的 `adaptive_gop` 与 `readiness` 分支。

7.4 论文的空间选择与时间配额

论文把运动矢量幅度和残差强度分别做稳健百分位归一，再融合 patch bitcost prior。选择单位从 OneVision 原型的单 patch 改为 2×2 patch block，与视觉塔 `spatial_merge_size=2` 对齐。

每组先保留一张完整 anchor canvas，其余预算形成 P canvases。若直接跨全部时间 global Top-K，一个运动极强的瞬间可能占满所有名额。论文 Equation 5、6 用分层时间配额为多个时间层保留容量，再在层内按分数选择，改善短事件覆盖。

当前公开 `codec-video-prep` 主链明确实现完整 anchor 加 2×2 block global Top-K，并有可选 event bins reserve。后者与论文目标相近，公式并不相同。当前主链分数还是 bitcost-only，旧 MV/residual scorer 虽保留在仓库，却不在 `run_preinfer()` 的默认调用链上。

【论文依据】Section 2.2、Equation 5、6、Figure 4。

【源码依据】`codec_selector/plugins/selectors/topk_2x2_bitcost.py::process_group_topk_2x2`；`codec_selector/codec_patch_gop/scoring.py::mv_res_score_map`。

7.5 共享视觉编码器与组内注意力

论文图示把图像视为一个时间组，普通抽帧按固定四槽分组，codec 输入按自适应逻辑 GOP 分组。视觉自注意力限制在组内，避免长视频所有 token 做全连接二次注意力。两层 connector 后的视觉 token 与文本一起进入 Qwen3。

发布模型使用 24 层视觉 Transformer、hidden 1024、16 heads、patch 14、merge 2 和 4:6:6 三轴 RoPE。_build_cu_seqlens() 根据 image_grid_thw 建立可变长度 attention 边界。某一行时间长度大于 4 时，模型按四 canvas 一块继续切分。

这意味着 release 中 codec group 的可见边界主要靠 canvas 顺序和固定 frame_windows_size=4 隐式表达。模型没有显式 group_ids tensor。

【源码依据】checkpoint remote code modeling_llava_onevision2.py 中的 LlavaOnevision2VisionRotaryEmbedding、_build_cu_seqlens 与 vision forward。

7.6 训练课程的论文设计

论文把 MLLM 训练分为四阶段。

阶段	主要数据与目标	视频表示
Stage 1 alignment	大规模图文对齐，加入约 420 万条短视频 caption	1 FPS，最多 30 帧
Stage 2 mid-training	指令、FineVision、30 至 180 秒视频	普通均匀帧，最多 60 或 90 帧
Stage 3 SFT	加入短视频指令和约 35 万条 10 至 15 分钟视频	普通均匀帧，最多 384 帧
Stage 4 codec curriculum	只把长视频 caption 子集切为 codec，混入空间和指向数据	384 或 768 codec 预算与标准表示混合

论文 Section 5 另写每步约 50% codec、37.5% uniform chunk、12.5% image。由于 Section 4 同时说明前三阶段使用标准帧，更稳妥的理解是这个比例对应最终 codec-capable 阶段或特定训练设置。公开脚本不足以进一步消歧。

【论文依据】Section 4、5。

7.7 官方实现分散在四处

本文读取的主要版本如下。

组件	审计提交	真实职责
EvolvingLMMs-Lab/ LLaVA-OneVision-2	5bede6462a0b321206c14a5982ace5c0455abd90	Megatron 训练类、Transformer、配置与脚本
HF checkpoint remote code	802f16c8346a062fef7ddf92b2bd64a770a50a1a	用户通过 trust_remote_code 实际加载的 processor 与 model
lmms-eval 专用分支	3997a60cb8e79d9341ac1e4a286f0bb739bcc779	论文精确评测 backend、参数和离线资产读取
codec-video-prep	77e8e91c11bb2fd520701e49465c9001f6c5b8ad	视频探测、bitcost、分组、Top-K 和 canvas

主 GitHub 仓的 processor 当前没有 codec 分支。真正的在线 codec 路径位于 HF 权重仓随 checkpoint 下发的 remote code。官方复现文档把 processor commit 5a75eaf7 作为版本锚点，wrapper 调用 from_pretrained 时却没有传 revision 或 code_revision，复现者需要自己固定。文档还要求 TestPyPI 的 codec-video-prep-legacy-exact==0.2.5.post2。该 wheel 源码没有随当前公开仓库提供，论文分数不能直接等同于 codec-video-prep main 的默认结果。

7.8 普通帧路径的真实调用链

Checkpoint 的 LlavaOnevision2Processor.__call__ 调用视频 processor。解码优先 Decord，失败后使用 OpenCV。choose_target_frames 按视频时长和 max_frames 决定帧数，小于 10 秒常取 8 帧，小于 30 秒常取 16 帧，更长视频受固定帧数或目标 FPS 约束。

每张帧走 Qwen2VLImageProcessor。build_patch_positions 用真实源 frame id 作为时间坐标，并把位置换成 2x2 block layout。文本中的一个 video 占位被改写成若干段 <t seconds> 和视觉 token。

video_preprocessor_config.json 虽写 max_frames=768，自定义 LlavaOnevision2Processor.from_pretrained 没有把该字段读进视频 processor，类内默认仍为 384。要得到 768 帧需要 per-call 参数或由 lmms-eval 外层采样控制。

Release 有一个与论文图示不同的行为。原始 `video_grid_thw=[T,H,W]` 被展开为 `T` 行 `[1,H,W]`。模型据此把每帧当独立视觉序列。官方 `lmms-eval` 的 `frame backend` 也把抽样帧作为多张 `images` 送入。公开 release 没有清楚落实论文所画的普通帧固定四槽跨帧注意力。

【源码依据】checkpoint `video_processing_llava_onevision2.py` 中的 `extract_video_frames_to_pil`、`choose_target_frames`、`build_patch_positions`；`processing_llava_onevision2.py` 的 `frame branch`。

7.9 Codec 推理的真实调用链

```
AutoProcessor.from_pretrained(..., trust_remote_code=True)
→ LlavaOnevision2Processor.__call__(video_backend="codec")
→ process_codec_video(video_url, CodecConfig)
→ 外部 cv-preinfer
→ canvas_*.jpg + src_patch_position.npy + meta.json
→ drop_padding_cansaves
→ Qwen2VLImageProcessor
→ 位置重排 + 时间戳文本重写
→ model.forward 或 model.generate
```

Checkpoint helper 的 `dataclass` 默认 `target_canvas=32`、`group_size=32`、`images_per_group=4`、`patch 14`、组长 8 至 64。其 `max_pixels` 字段表面默认 150,000，processor 构造时会继承 `image processor` 的预算。Checkpoint `preprocessor_config.json` 写 4,000,000，未显式覆盖时 `codec` 有效默认因此是 4M。官方 `run_codec.sh` 才显式设为 313,600。候选帧数按下式计算。

$$N_{\text{sample}} = \text{target_canvas} / \text{images_per_group} \times \text{group_size}$$

默认得到 256。Helper 实际调用 `cv-preinfer` 时固定 `grouping_mode=readiness`，传 `group size`、每组画布、`patch`、像素预算、组长和规避关键帧。它没有传固定 `canvas` 数，也没有打开论文 `adaptive GOP`。`target_canvas` 主要影响候选数量和缓存键，不保证输出刚好 32 张。

【源码依据】checkpoint `codec_video_processing_llava_onevision2.py` 中的 `CodecConfig`、`process_codec_video` 与命令构造。

7.10 Payload、padding 与时间戳

`_load_codec_result()` 的实际 `payload` 包含 `canvas` 图片列表、`N×3 src_positions`、`fps`、输出目录和 `meta`。每行位置是 `[source_frame_id, patch_h, patch_w]`。论文 U 中的 `canvas index`、`packed coordinate` 和 `group id` 没有全部作为独立 `tensor` 送入模型。

`drop_padding_canvases` 只删除整张 canvas 都为负时间位置的 padding。若同一 canvas 一部分有效、一部分 padding，代码直接报错。离线资产必须以 canvas 为粒度补齐。

`codec_positions_for_processor` 校验位置数量恰好等于 $\Sigma(T \times H \times W)$ ，随后换成 2x2 merge 顺序。

`rewrite_text_with_codec_positions` 把连续相同 source frame 的 patch run 写成对应 `<x.x seconds>` 段，并生成匹配数量的 image pad token。

视觉时间因此有两条通道。Patch 级 source frame 进入 3D RoPE，秒数文本直接进入 Qwen3。前者服务视觉相对位置，后者让语言模型可明确回答时间。

7.11 Canvas 合并修复为何重要

Codec canvas 已按 patch 网格准备。若 image processor 再 smart resize，位置数量会错位。Helper 会调整 image processor 的 min/max pixels，尽量保持原 canvas 尺寸。

早期 processor 为每张 canvas 生成一行 `[1, H_p, W_p]`，视觉注意力会把 canvas 彼此隔离。当前 processor 将同一视频的同尺寸 canvas 合成一行 `[N_canvas, H_p, W_p]`。模型再按每四张切一个局部窗口。官方注释记录，这项修复在 VideoEval-Pro 上约恢复 1 点。

这也是为什么评测必须记录并主动固定 remote-code revision。模型权重不变，processor 如何构造 `image_grid_thw` 也会改变视觉注意力拓扑。

【源码依据】checkpoint `processing_llava_onevision2.py` 的 codec grid 聚合；官方评测 README 固定的 processor revision。

7.12 核心张量怎样流过模型

令 merge 前 patch 总数为 P，空间 merge 为 2。

字段	实际形状	含义
<code>input_ids</code>	<code>[B,L]</code>	文本、时间戳和视觉占位
<code>attention_mask</code>	<code>[B,L]</code>	语言序列 mask
<code>pixel_values</code>	<code>[P,588]</code>	每行展平的 $3 \times 1 \times 14 \times 14$ RGB patch

字段	实际形状	含义
patch_positions	[P,3]	[source_t,source_h,source_w], 已按 block layout 排列
image_grid_thw	codec 常为 [N_canvas,H_p,W_p] 一行	定义 varlen attention 边界
vision hidden	[P,1024]	patch embedding 与 24 层 ViT 输出
merged features	[P/4,4096]	2×2 merger 与两层投影后

源码注释有时把 pixel_values 写成 4D，模型 get_image_features() 实际只接受 2D，并在 patch embedding 前 view(-1,3,14,14)。应以可执行代码为准。

patch_positions 在函数签名中标为 optional，vision forward 却无条件调用 forward_from_positions。实际缺失会失败，这是接口注解与真实契约不一致。

Merger 将每四个相邻 patch 的 1024 维特征拼接，LayerNorm 后通过两层 MLP 投影为 4096。主模型找到所有 <|image_pad|> 的 embedding 位置，用这些视觉特征替换，再交给 Qwen3 的 36 层因果解码器。生成进入 KV cache 后，后续 token 不再重复传视觉张量和重跑 ViT。

【源码依据】checkpoint modeling_llava_onevision2.py 的 patch embedding、vision forward、merger、get_image_features、主 forward 与 prepare_inputs_for_generation。

7.13 训练链与在线推理的不同

训练时不在 dataloader 内执行 cv-preinfer。Codec 样本已经离线变成 raw_image canvas 列表、raw_patch_positions、fps 和时间戳 metadata。Qwen2VLTaskEncoder.process_sft_qa 把它们统一走 image path，合并 grid，换成 block layout，再构造语言占位。

训练模型执行 images → vision model with positions → adapter → 替换图像占位 → Qwen3。pixel_values_videos 分支直接抛 NotImplementedError，因为模型边界上的“视频”已经表示为 image-like canvases 或 frames。损失仍是标准 causal next-token loss，没有 codec 重建损失或额外稀疏正则。

公开主仓给出核心类和若干训练 shell，数据 YAML 含内部路径。Stage 1 至 4 的完整数据配方、codec 资产生成命令和训练日志尚未形成可一键执行的复现流程。

【源码依据】主仓 qwen2vl_task_encoder.py::process_sft_qa；task_encoder.py；llava_onevision2_model.py；pretrain_llava_onevision2.py。

7.14 论文算法与 release 的差异

技术点	论文提出	当前公开 release	判定
重要性分数	MV + residual + patch bitcost	当前公共主链为 bitcost-only，旧 MV/res 模块不在入口链	部分对应
自适应 GOP	P/B 累计配额、min/max、局部谷值	默认 readiness，可选单帧分位数切分	算法不同
分层时间配额	Equation 5、6	global Top-K，可选 event bins reserve	目标相近，公式不同
Dense anchor	一张完整，其余稀疏	process_group_topk_2x2 明确实现	一致
2x2 block	与 merge 对齐	默认 block 2	一致
显式 group id	U 含 κ	模型只收 [t,h,w]，组由 canvas 顺序隐式表示	字段缩减
Codec 组注意力	自适应组内可见	每连续四 canvas 可见	固定化
普通帧四槽	论文图示为四帧组	release 展成每帧独立行	未清楚实现
四阶段训练	完整 curriculum	核心类公开，完整 recipe 与资产脚本缺失	部分公开

7.15 实验结果应怎样解释

论文的 codec 优势集中在短暂事件检索和时间定位。Appendix Table 4 给出 JumpScore mAP 随预算的对照。

Canvas 预算	Uniform	Codec
4	32.5	39.4
8	35.2	40.2

Canvas 预算	Uniform	Codec
16	36.7	46.9
32	37.6	58.3
64	39.9	71.3
128	45.4	74.9

论文报告 JumpScore 平均提升 17.3 点，temporal grounding 平均提升 9.7 点。它也给出反例。需要连续观察的 dense trajectory 和未来预测任务，均匀帧可能更好。Codec 选择保留变化证据，却会丢失平滑轨迹中的低分中间状态。

这些数字依赖论文模型、processor、预算和精确评测环境。官方复现还固定了特定 legacy-exact wheel，因此不能把表中差值外推到当前 main 的任意配置。

【论文依据】Section 8、Figure 7、8、Table 3，Appendix Table 4。

7.16 工程限制

- Codec helper 没有自动回退到普通帧 backend，外部命令失败会中止当前处理。
- 缓存键使用视频路径字符串和配置，不哈希文件内容，同一路径替换文件可能命中旧缓存。
- target_canvas 在 readiness 下不是硬保证，短视频可少于目标，动态组也可能产生不同数量。
- Group id 没有显式 tensor，canvas 丢失、乱序或不是四的倍数会改变隐式分组。
- spatial_mask_mode 进入配置与缓存键，checkpoint helper 没有传到底层 CLI。
- Canvas 默认 JPEG quality 95，会发生一次额外有损压缩。
- 可变帧率视频若只用 frame id 除以 fps，时间戳可能偏移。

7.17 LLaVA-OneVision-2 的准确定位

LLaVA-OneVision-2 继承了 OneVision-Encoder 的视觉骨干、三轴位置和稀疏 RGB 原则，并新增 2x2 merger 对齐、时间戳文本、局部视觉窗口与 Qwen3 接口。论文提出的自适应逻辑 GOP 和多信号 saliency 比当前公开 main 更丰富。Release 则选择了更易部署的 bitcost-readiness 管线，并把组信息压缩成 canvas 顺序。

因此复现报告要同时给出模型 commit、remote-code processor commit、预处理包和参数。只写“使用 LLaVA-OneVision-2 codec 模式”不足以唯一确定实际算法。

第八章 Mage-ViT 与 Mage-VL 走向跨 codec 和流式交互

8.1 从离线稀疏视频到持续感知

Mage-ViT 延续 OneVision-Encoder 的 codec-aligned sparsity，以 16×16 patch、共享 3D RoPE 和从头训练的 24 层 ViT 处理稀疏视频。Mage-VL 再把视觉塔接到两层投影与 Qwen3-4B，并增加 cognition gate，决定连续视频里何时值得启动昂贵语言生成。

论文把贡献分成两层。

- 视觉层用 codec 码率分配选择动态和难预测区域，在相同 token 预算下覆盖更多时间。
- 交互层持续观察滚动窗口，gate 输出 p_speak ，超过阈值才触发 System 2 生成。



【论文依据】Mage-VL Section 3、4，Figure 2、3。

8.2 论文中的 Mage-ViT

Mage-ViT 把输入切成 16×16 patch，为每个时空位置构造重要度 $S[t, h, w]$ 。传统 HEVC 路径中，论文定义 S 为运动矢量幅度和 P 图片残差能量的加权组合。神经 codec DCVC-RT 路径直接使用概率模型的负对数似然。

$$S(t,h,w) \approx -\log_2 p(z_{t,h,w})$$

一个 latent 的预测概率越小，估计码长越高，说明它越难预测。选择器完整保留 I 或 anchor patch，再从预测图片按重要度取 Top-K。

论文示例使用 256×256、patch 16，每帧 16×16，共 256 个 patch。64 帧密集输入有 16,384 个 patch，预算 B 为 4,096，减少约 75%。这个口径与后文 tc32 从 256 个源帧打包约 32 canvases 的约八分之一工作量不同，不能混成一个数字。

视觉主干含 24 层 pre-norm Transformer、hidden 1024、16 heads、4 倍 GELU MLP 和 FlashAttention2。训练批次允许可变 token 长度，padding mask 阻止注意力进入无效位置。共享 3D RoPE 使用未剪枝网格上的源 (t,h,w) 。

【论文依据】Section 3.1、Figure 2。

8.3 Mage-ViT 的两阶段预训练

论文称 Mage-ViT 从头训练。

Stage 1 只用图像，分辨率 224 至 448，长宽比覆盖 1:2 至 2:1。优化器 AdamW，初始学习率 1e-3，余弦衰减、warm-up、gradient clipping 1.0 和 bf16。目标沿用大规模 cluster discrimination，先由 MetaCLIP 提特征和 K-means 原型，再做负采样分类。

Stage 2 联合图像和视频。视频使用 256²、64 帧、4,096 token，codec、chunk-wise、collage 的比例约为 50%、40%、10%，学习率降到 5e-5。

论文摘要写约 5.6 亿张无标签图像和 1 亿视频帧。公开独立 Mage-ViT 模型卡仍有历史数据量口径，报告应以论文 v1 为准并注明版本差异。

【论文依据】Section 3.2。

8.4 Mage-VL 的五阶段课程

Mage-VL 以 Qwen3-4B-Instruct-2507 为语言骨干。论文前四阶段沿用 LLaVA-OneVision-2 的结构设计，逐步完成图文对齐、混合中训、长视频 SFT 和 codec-native 长上下文适配。Stage 4 把大量长视频转成含完整 anchor 与稀疏更新的 rolling token windows。

Stage 5 冻结视觉骨干、EPFE 和基础 LLM，只训练 cognition gate。约 335 万条 streaming 样本提供 silent/speak 目标。Caption 起始时间作为 speak，其他候选时间作为 silent，并使用类别加权 token 交叉熵处理 speak 稀少问题。Gate 打开时，论文设计把最近 N 个 codec 段交给冻结 VLM 做标准自回归生成。

【论文依据】Section 4.2、4.3、Figure 5。

8.5 需要同时读取的发布物

工件	审计提交	主要内容
microsoft/Mage GitHub	76bec2bb3818863f470de7e867c2dc7f1d0bfd83	入口、README、普通推理与 streaming demo
HF microsoft/Mage-VL	d88b153285f1633a61b2f693c59c8576693af185	Processor、视觉塔、Qwen3、gate、traditional/DCVC 适配
HF microsoft/Mage-ViT	de3bd807f06fc9e8045540041dad3bf429a1eae7	独立视觉编码器配置、模型代码与权重
codec-video-prep	77e8e91c11bb2fd520701e49465c9001f6c5b8ad	Traditional codec 的直接运行时依赖

GitHub 主仓没有 processor、视觉塔和 gate 核心。inference_base.py 以 trust_remote_code=True 从 HF 模型仓加载它们。只审计 GitHub 会漏掉真正的数据流。

HF 发布物在论文之后修复过 image/video placeholder 对齐。下文描述当前官方发布代码，不自动代表论文表格使用的内部快照。

8.6 Traditional codec 的真实入口

mage_vl/inference_base.py::run_offline 把 codec_engine=traditional 映射成字符串 engine=hevc，将 --num-frames 复用为 target_canvas，显式设 patch 16，再调用 processor 的 codec backend。

MageVLProcessor.__call__ 进入 process_codec_video。Traditional 分支 _run_cv_preinfer 启动外部命令，实质参数如下。

```
cv-preinfer
--num_sampled_frames 256
--grouping_mode readiness
--group_size 32
--images_per_group 4
--patch 16
--max_pixels 150000
--min_group_frames 8
--max_group_frames 64
--avoid_keyframes
--canvas_format jpg
```

engine=hevc 没有触发转码，也没有强制源流必须是 HEVC。codec-video-prep 会探测实际输入，当前支持 H.264、HEVC 与 VP9 bitcost。这个字符串只代表 Mage 的 traditional backend 标签。

target_canvas=32 没有作为硬约束传给 cv-preinfer。它只通过 $(32/4) \times 32$ 推出 256 个候选帧。

Readiness 组长可在 8 至 64 变化，最终 canvas 数可能高于或低于 32。论文 Table 2 中 HEVC 平均约 33.4 canvases 与这种动态结果相符。

命令显式启用 --avoid_keyframes。完整保留的 anchor 是每个 readiness 组首个采样帧，通常并非真实 I 图片。论文图中的 I-frame 或 codec keyframe 在解释当前公开默认链时应改称逻辑全帧锚点。

【源码依据】GitHub mage_vl/inference_base.py::run_offline；HF codec_video_processing_mage_vl.py::process_codec_video、_run_cv_preinfer。

8.7 论文的 MV 加 residual 与代码的 bitcost

Traditional 入口继续调用 codec-video-prep。

```
cv-preinfer
→ codec_video_prep.cli.main
→ codec_video_prep.api.run_preinfer
→ run_preinfer_config
→ run_bitcost_readiness
```

因此当前可执行链使用块级编码 bitcost，随后 readiness 分组和 2×2 global Top-K。公开入口没有分别提取 MV 数组和 residual 数组，也没有显式加权融合。运动与预测误差会通过编码器的语法成本间接影响 bitcost，信号形式已经聚合。

论文中的 HEVC 选择器属于方法定义，当前发布 traditional 推理属于动机相同的工程替代。报告若写“当前 Mage 把 MV 与 residual tensor 送进 ViT”，会同时错两次。当前选择器读 bitcost，ViT 读 RGB patch。

【论文依据】Section 3.1、Figure 2。

【源码依据】mage_vl/requirements.txt；codec_video_prep/api.py::run_preinfer_config；codec_selector/core/pipeline.py::run_bitcost_readiness。

8.8 从视频到 Qwen3 的张量链

设实际输出 K 张 canvas，每张按 16×16 切成 H_p×W_p，合并前 patch 总数 N=K×H_p×W_p。

阶段	张量	形状
Canvas	RGB JPG	K × 16H _p × 16W _p × 3
来源位置	src_patch_position	[N,3]
Image processor	pixel_values	[N,768]，每行是 3×1×16×16
Processor	image_grid_thw	[K,3]，每行为 [1,H _p ,W _p]
Processor	patch_positions	[N,3]，2×2 block 顺序
Patch embedding	hidden	[1,N,1024]
24 层 Mage-ViT	hidden	[1,N,1024]
Merger 与 projector	visual tokens	[N/4,2560]
LLM 输入	inputs_embeds	[B,L _{text} +N/4,2560]
Qwen3 输出	logits	[B,L,151936]

实际调用链如下。

```
MageVLForConditionalGeneration.generate
→ MageVLModel.forward
→ get_image_features
→ MageVLVisionPretrainedModel.forward
```

```
→ MageVLVisionEmbeddings
→ 24 × MageVLVisionEncoderLayer
→ MageVLVisionPatchMerger
→ get_placeholder_mask / masked_scatter
→ Qwen3 language_model
→ lm_head
```

`prepare_inputs_for_generation` 在 `prefill` 后清空 `pixel_values`，后续文本 token 复用 KV cache，不会重复运行视觉塔。

【源码依据】HF `modeling_mage_vl.py` 中的生成、模型前向、视觉前向、merger 和 placeholder 注入。

8.9 Patch embedding、3D RoPE 与注意力边界

Checkpoint 视觉配置为 patch 16、24 层、hidden 1024、MLP 4096、16 heads、merge 2、frame window 4。`MageVLVisionEmbeddings.forward` 把 `[N,768]` reshape 为 `[-1,3,16,16]`，每个小图经 kernel 与 stride 都为 16 的 Conv2d，产生一个 1024 维向量。

3D RoPE 每 head 64 维，half dimensions 为 32，按 $T:H:W = 4:6:6$ 分为 8、12、12，再复制到 64。它直接使用来源 frame id 与 patch 坐标。

当前 processor 把普通视频每帧和 codec 每张 canvas 都表示为独立 `[1,H,W]` 行。

`_build_cu_seqlens` 按行建立 sequence。因此公开路径中的视觉 self-attention 每帧或每 canvas 独立，不会跨所有 canvases 全局交互。同一 canvas 内可混入多个源时刻的 patch，它们可以彼此注意；不同 canvas 的组合主要交给 Qwen3 和 timestamp token。

3D RoPE 只影响同一 attention window 内的相位。两个 token 若被 `cu_seqlens` 分到不同窗口，RoPE 不会让它们跨窗通信。

【源码依据】HF `modeling_mage_vl.py` 中的 `MageVLVisionEmbeddings`、`VisionRotaryEmbedding`、`_build_cu_seqlens`。

8.10 Merger 同时承担 projector

`MageVLVisionPatchMerger` 先把连续四个 1024 维 patch 拼成 4096，随后执行

```
LayerNorm
→ Linear(4096,4096)
→ GELU
→ Linear(4096,2560)
```

它同时完成 2x2 spatial merge 和论文中的两层 vision-language projector。公开代码没有另一个独立 Projector 类。

主模型用 `masked_scatter` 把 $[N/4, 2560]$ 视觉特征写到 `<|image_pad|>` 的词嵌入位置。Qwen3 内部继续使用普通 1D position ids。时间位置在视觉塔由 3D RoPE 表达，在 LLM 里靠 timestamp block 与 token 顺序表达。

8.11 DCVC-RT 路径只替换评分源

DCVC 分支随 HF 模型仓携带 neural codec 和一套 vendored 或 forked 的 readiness、Top-K、canvas 工具。`dcvc_readiness_gen.py` monkeypatch 两个函数。原来的 `bitcost fetch` 只转交 frame id，`score-map` 函数改为顺序运行 DCVC-RT，返回 learned rate bitmap。抽帧、readiness、2x2 block 选择、打包和 $N \times 3$ 坐标格式保持同一代码谱系。

`_gaussian_bits` 用量化值与概率模型标准差估计每个量化 bin 的概率质量，再取 $-\log_2(\text{prob})$ 。它没有真的调用算术编码器写 bitstream，得到的是概率码长估计，足以用于排序。

DCVC engine 顺序维护 decoded picture buffer。第一个时刻走 intra，其余走 inter，重建 `x_hat` 只用于保持下一步预测状态。最终 RGB canvas 仍从原输入视频的解码帧裁 patch，不从 DCVC 的 `x_hat` 裁。因此 HEVC 与 DCVC 对比主要改变选择地图和 canvas 数量，ViT 看到的像素来源保持一致。

【源码依据】HF `neural_codec/dcvc_readiness_gen.py`；`neural_codec/dcvc_rt_engine.py`。

8.12 独立 Mage-ViT 与集成视觉塔

HF `microsoft/Mage-ViT` 是纯视觉 checkpoint，不包含 codec parser、选择器和 canvas 生成。它接收图像或视频 dense tensor，也可接收 `visible_indices` 或显式 `patch_positions`。Sparse gather 发生在 Conv patch embedding 之后、Transformer 之前。它能节省主干 attention 和 MLP，仍会为 dense 输入执行全部 patch convolution。

独立模型对一个样本的可见 token 做全局非因果 attention，末端还有 learned-probe attentive pooling，输出 patch 特征和 1024 维 pooled 表示。

Mage-VL 集成视觉塔接收 processor 展平的 patch rows，使用 `cu_seqLens` 分段，末端没有 pooling head，而是 2x2 merger 与 2560 维投影。Codec 选择发生在 processor 之前，连 dense patch convolution 也可以省掉。

【源码依据】HF Mage-ViT `modeling_mage_vit.py::MageViTModel.forward`；模型卡对上游 selector 的说明。

8.13 Gate 的公开结构

`streammind_gate.py` 的网络数据流如下。

```
vision tokens [B,T,P,2560]
  → 对 P 做空间平均 [B,T,2560]
  → Linear + LeakyReLU
  → 1 层 Mamba SSM + LayerNorm
  → Linear + LeakyReLU
  → 4 层 Qwen3 二分类器
  → [B,T,2] silent / speak logits
```

二分类 Qwen3 使用 hidden 2560、32 query heads、8 KV heads、intermediate 12288、4 层。源码损失权重为 `[0.15,0.85]`，提高 speak 类权重。Gate 权重文件约 1.073 GB，若主要是 BF16，对应约 5.37 亿参数。“轻量”应理解为相对完整 4B VLM 较轻，并非微型线性头。

`MageVLModel._streammind_vision_tokens` 先跑共享视觉塔，得到 merge 后 2560 维 token，再按时间和每时刻 patch 数 reshape 为 `[1,T,P,2560]`。`streammind_gate_forward_segments` 分别编码 segment 后沿 T 拼接，让 Mamba 看到因果序列。

【源码依据】HF `streammind_gate.py`；`modeling_mage_vl.py` 中的 `_streammind_vision_tokens`、`streammind_gate_forward_segments`。

8.14 公开 streaming demo 的实际边界

论文 Section 4.1、4.3 描述到达即处理的 codec windows、累积 streaming memory 和生成时最近 N 段滑窗。GitHub `inference_streaming.py` 当前执行另一种离线演示。

1. 默认按 8 秒用 `ffmpeg -c copy` 切片，切点可能回退到附近关键帧。
2. 循环先构造并预处理完整文件的全部 segments。
3. 所有 segment 一次传给 `streammind_gate_forward_segments`。
4. 读取每个 segment 边界的 speak probability。

5. Gate 打开后，生成函数只接收当前 segment，没有拼最近 N 段。

Mamba 计算本身是因果的，理论上可以逐段在线推进。脚本没有暴露持久化 inference state、摄像头或网络流输入、逐段到达 API，也没有实现任意时刻插入用户 query。公开系统可验证 gate 网络和因果演示，尚不能视为论文完整实时服务实现。

【源码依据】GitHub mage_vl/inference_streaming.py 的切片、segment 循环、gate 调用与 generate_current_segment。

8.15 跨 codec 结果

论文 Table 2 在不做 codec-specific retraining 的条件下，把训练时 HEVC 选择器换成 DCVC-RT。代表性结果如下。

任务	HEVC 分数	HEVC 平均 canvases	DCVC 分数	DCVC 平均 canvases
NExT-QA	83.1	32.7	82.6	32.1
VideoMME	64.0	33.2	64.3	30.2
LVBench	41.8	32.2	43.5	29.6
TimeLens Charades	50.7	33.5	50.5	31.9

结果支持一个谨慎结论。只要不同 codec 都把预测困难映射为相近的局部码率结构，同一视觉模型有机会跨 codec 工作。它没有证明任意码率、任意转码器或任意神经 codec 都可无损替换。

论文 Table 5 还报告 NExT-QA 在一个对照下由 1,460 秒降到 415 秒，约 3.5 倍 wall-clock 加速，同时得分由 79.8 上升到 80.8。这个数字来自论文基准环境，公开仓库未提供完整一键 benchmark harness。

8.16 论文与公开代码对照

技术点	论文提出	当前公开实现	判定
HEVC 重要度	MV magnitude + residual energy	block bitcost → score map	动机相同，信号实现不同
全帧底座	完整 I frame	Readiness 组首逻辑 anchor，且避开真实 keyframe	结构相近，术语不同
Codec 输入	HEVC 默认，DCVC 可替换	Traditional 处理实际源 codec，DCVC 独立 bundled	engine=hevc 不强制转码

技术点	论文提出	当前公开实现	判定
ViT 内容	Codec-native sparse tokens	RGB patch + 源坐标, codec 只做选择	已核验
3D RoPE	保留源 T/H/W	patch_positions 直接进入视觉 RoPE	已实现
跨帧视觉注意力	论文强调时空上下文	当前每 frame/canvas 独立窗口	时间组合主要在 LLM
两层 projector	独立概念	融入 patch merger MLP	功能已实现
两阶段 ViT 训练	数据与目标详述	只有权重、配置和推理模型	训练未开源
五阶段 VL 训练	课程与冻结方案	没有 trainer 和数据管线	无法源码复现
Streaming	增量 memory, 最近 N 段生成	Gate 网络开源, demo 全片预处理并只生成当前段	系统未完整产品化

8.17 复现时最容易踩的坑

- Checkpoint 视觉 patch 为 16, 部分配置类和 codec config 默认仍写 14。官方入口会覆盖, 直接调 processor 时必须显式检查。
- 依赖只写 codec-video-prep>=0.2.5, 没有上界和 commit。依赖升级可能改变 canvas。
- Cache key 不含 codec-video-prep、Ffmpeg 和 cv_reader 版本, 升级后可能复用旧资产。
- spatial_mask_mode 进入配置和 cache key, 当前 traditional 命令没有传给选择器。
- target_canvas 是 nominal budget, 不是严格结果数量。
- Streaming 的 ffmpeg -c copy 切片受关键帧 seek 影响, 实际段边界可能偏移。
- 发布后有 placeholder 修复, 模型代码 revision 会影响图文混合输入对齐。

8.18 Mage 的准确定位

Mage-ViT 没有直接理解压缩 bitstream。Codec 在视觉编码之前提供预算先验, canvas 把不规则 patch 装回成熟图像栈, 源坐标纠正装箱后的时空身份, 视觉塔提取语义, Qwen3 跨 timestamp blocks 推理, gate 决定何时启动生成。

论文进一步提出显式 MV 与 residual HEVC 选择器、完整五阶段训练和真正的在线滑窗系统。当前发布代码完整公开了推理张量链、DCVC rate-map 替换与 gate 网络, 却没有公开这些训练与服务环节。它已经是可运行研究原型, 距离论文描述的持续在线系统仍有明确工程缺口。

第九章 把五层系统连成一条数据流



阶段	输入	主要工作	输出
Streaming gate	滚动视觉特征	Silent/speak 决策	是否启动生成

Codec-native 只描述第二阶段的预算原则和它与视觉位置接口的配合。系统仍会解码被选区域的 RGB，仍要运行视觉编码器和语言模型。它减少的是重复视觉证据，不会让整个视频绕过解码。

9.2 每项工作改变了哪一层

OneVision-Encoder

它提出视觉原语。Codec 信号选择稀疏 RGB patch，紧凑打包维持高效张量，源坐标 3D RoPE 修复打包后的时空位置。官方代码进一步展示了 gather 后重排成规则伪视频的办法。

codec-video-prep

它把选择器工程化。Patched FFmpeg 取得 H.264、HEVC、VP9 的局部 bitcost，公开单视频链增加 readiness，JSONL 链保留 packet energy 和 mvres 路线。Canvas 与 N×3 position 成为模型无关资产。

LLaVA-OneVision-2

它把稀疏视觉接入 Qwen3-8B，加入 2×2 merger 对齐、时间戳文本和局部视觉窗口。论文设计了 P/B cost 配额和局部谷值自适应逻辑 GOP，release 则主要使用外部 bitcost-readiness。

Mage-ViT 与 Mage-VL

它把 patch 改为 16，从头训练自己的视觉编码器，并把传统 codec 与 DCVC 概率码长放进同一选择骨架。Mage-VL 使用 Qwen3-4B、滚动视觉段和 silent/speak gate，公开 demo 仍是离线因果演示。

9.3 设计维度总表

维度	OneVision-Encoder 论文与核心代码	codec-video-prep 公开链	LLaVA-OneVision-2 release	Mage-VL release
视觉 patch	14	默认 14，可配置	14	16
选择信号	论文 MV+res，公开训练资产有多条路径	Bitcost	Bitcost-readiness 主线	Traditional bitcost，DCVC 概率码长
时间候选	64 帧固定 clip	默认 1024 均匀候选	目标 canvas 反推候选	目标 canvas 反推候选

维度	OneVision-Encoder 论文与核心代码	codec-video-prep 公开链	LLaVA-OneVision-2 release	Mage-VL release
分组	论文固定 GOP，代码多种口径	Readiness、fixed、内部 adaptive	每四 canvas 视觉窗口，组信息隐式	每 canvas 独立视觉窗口
稀疏单元	单 patch global Top-K 实现	2×2 block	2×2 block	2×2 block
Anchor	论文完整 I，代码路径不统一	组首或 adaptive anchor	完整逻辑 anchor	完整逻辑 anchor，默认避开 keyframe
来源位置	visible_indices 或 [t,h,w]	N×3 [frame,h,w]	[t,h,w]	[t,h,w]
视觉骨干	24L、1024、16 heads	无模型	24L、1024、14，接 Qwen3-8B	24L、1024、16，接 Qwen3-4B
Streaming	无	预处理工具	论文列为未来	论文有 gate，公开 demo 未完整在线化

9.4 名称相同，语义可能不同

名称	可能含义	检查方法
GOP	码流预测结构、packet-energy 逻辑组、readiness 组	看它在编码时还是预处理时生成
Keyframe	IDR/CRA、MP4 同步样本、视觉代表帧	看来源是 decoder、container 还是镜头算法
I-frame canvas	真实 I 图片、第一张 sampled frame、逻辑组首 anchor	追踪 anchor 选择函数和 avoid_keyframes
Codec token	压缩语法、由 codec 选择的 RGB patch、merge 后视觉 token	检查 ViT 输入 dtype 和 channel
Target canvas	严格数量、nominal 预算、反推候选数的参数	查参数是否传到实际分组函数
3D 位置	Canvas 坐标、源 [t,h,w]、LLM timestamp 文本	分别检查 position array、RoPE 与 prompt

9.5 误差怎样沿链条传播

这套系统最危险的错误常常不会触发 shape exception。

若 frame id 与 RGB 槽位错位，选择器仍能输出合法索引。若 block-raster 与 processor merge 顺序错位，位置数量仍可能相等。若 canvas 顺序变化，隐式 group 仍是四张一组。模型会正常前向，却把内容绑定到错误时间或空间。

应优先建立几何不变量。

- 每个来源位置都能映射回一个有效 frame 和 patch 网格。
- `len(src_positions)` 等于 merge 前 $\Sigma(T \times H \times W)$ 。
- Block layout 中每四行位置属于同一 2x2 源 block，顺序稳定。
- 合并后视觉特征数等于 prompt 中 image pad 数。
- 同一 source frame 的 timestamp 与 frame id、fps 或 PTS 一致。
- Padding position 不会进入正常视觉 token。

9.6 计算收益出现在哪里

总延迟可以写成

$$T_{\text{total}} = T_{\text{probe}} + T_{\text{decode}} + T_{\text{score}} + T_{\text{select}} + T_{\text{vit}} + T_{\text{llm}}$$

Codec-native 主要降低 T_{vit} ，并通过减少投影后视觉 token 降低部分 T_{llm} 。 T_{decode} 和 T_{score} 会增加或改变。默认 `cv_reader_pixels` 把 RGB 与 bitcost 放进一次扫描，努力控制前处理开销。

若使用独立 FFmpeg 像素解码和 `cv_reader` 成本扫描，视频可能被读取两次。

视觉 token 减少不等于端到端同比加速。短视频上固定启动、磁盘缓存和解码可能占主导。长视频和高分辨率输入更容易从视觉计算削减中受益。Mage 论文的最高 3.5 倍属于特定任务、预算和硬件下的端到端结果，不能直接用 75% token reduction 推导。

9.7 什么时候适合使用

适合的任务通常关注短暂事件、时间定位、粗粒度动作、长视频问答和持续监控。视频包含大量静止背景时收益更明显。

需要谨慎的任务包括细密轨迹、手部微动作、逐帧计数、低运动静态文字、未来状态预测和对均匀时间连续性高度敏感的分析。此时应保留更多均匀帧或使用混合选择。

第十章 如何评测、复现和部署

10.1 不能只报告任务分数

一套公平评测至少应同时覆盖效果、预算、延迟和稳健性。

维度	建议指标
任务效果	QA accuracy、temporal grounding、tracking、caption quality
视觉预算	源候选帧数、canvas 数、merge 前 patch、merge 后 token
前处理	Probe、decode、score、group、pack 的独立耗时
模型	ViT prefill、LLM prefill、decode tokens/s、峰值显存
端到端	冷缓存与热缓存 wall-clock、吞吐、首 token 延迟
稳健性	H.264/HEVC/VP9、码率、GOP、转码器、分辨率、VFR
选择质量	事件召回、时间覆盖、被选 block 回投可视化

Uniform baseline 必须匹配最终视觉 token 或计算预算。只匹配源帧数量会让 codec 路线看起来天然更贵或更便宜。还要区分冷缓存首次生成和热缓存重复推理。

10.2 版本记录模板

复现日志至少保存下列字段。

```
paper_version
model_repo_commit
processor_remote_code_commit
codec_video_prep_commit
ffmpeg_build_and_patch
cv_reader_build
source_video_codec_profile_bitrate_gop
all_preprocess_arguments
canvas_hashes
position_array_shape_and_hash
model_config
hardware_and_dtype
```

这份清单看起来繁琐，原因很实际。LLaVA 的 processor grid 聚合修复会改变视觉窗口，Mage 的 HF 发布物在论文后修复过 placeholder 对齐，codec-video-prep>=0.2.5 又没有固定上界。同一模型权重搭配不同 remote code 或预处理版本，可能得到不同输入拓扑。

10.3 建议的单元测试

编码与采样

- 构造已知 IDR 间隔的短视频，核对 MP4 stss 和 ffprobe 路径得到的 frame ids。
- 对 uniform_count、fps、packet peak 分别保存候选列表，当前提交特别检查 pkt_peak 枚举不匹配。
- 用含 B 图片的视频核对显示时间和 frame id，不把 packet 顺序当显示顺序。

几何与打包

- 用彩色编号 patch 合成视频，经过 packer 后逐块反投影，要求像素和 [frame,h,w] 全部一致。
- 检查右下 padding 不改变左上源坐标。
- 检查 raster、block-raster 和 processor merge 三种顺序互相可逆。

模型接口

- 校验 $P = \text{len}(\text{patch_positions}) = \text{pixel_values.shape}[0]$ 。
- 校验 merger 后数量 $P/4$ 等于 image pad token 数。
- 打乱 canvas 顺序应使显式测试失败，防止隐式 group 被静默改变。
- Batch 大于 1 时使用不同位置图，确认 wrapper 不只取第一个样本。

Streaming

- 逐段与一次性 gate 前向的边界 logits 应在相同数值容差内。
- State reset 后不同视频不能共享 Mamba memory。
- 生成窗口应明确包含当前段和最近 N 段，记录淘汰策略。

10.4 当前 codec-video-prep 的安全调用示例

下例避开审计提交中的 pkt_peak 名称问题，显式使用 uniform_count 与 readiness。

```
codec-video-prep \
  --video input.mp4 \
  --out_dir output_codec \
  --num_sampled_frames 256 \
  --frame_sampling_mode uniform_count \
  --grouping_mode readiness \
  --group_size 32 \
  --images_per_group 4 \
  --min_group_frames 8 \
```

```
--max_group_frames 64 \  
--patch 14 \  
--block_size 2 \  
--decode_backend cv_reader_pixels \  
--canvas_format png
```

Python API 可以直接得到结果对象。

```
from pathlib import Path  
  
import numpy as np  
from codec_video_prep import run_preinfer  
  
result = run_preinfer(  
    video="input.mp4",  
    out_dir="output_codec",  
    num_sampled_frames=256,  
    frame_sampling_mode="uniform_count",  
    grouping_mode="readiness",  
    group_size=32,  
    images_per_group=4,  
    patch=14,  
    block_size=2,  
    decode_backend="cv_reader_pixels",  
    canvas_format="png",  
)  
  
print(result.canvas_files)  
positions = np.load(Path(result.out_dir) / "src_patch_position.npy")  
print(positions.shape)
```

审计提交的 PreinferResult 字段以实际 api.py 为准。meta.json 的 decode_backend 存在硬编码问题，调试时继续保存调用参数。

10.5 缓存与资产治理

缓存键应包含视频内容 hash、全部配置、预处理仓 commit、patched FFmpeg 标识、processor revision 和输出 schema 版本。LLaVA 与 Mage 当前 helper 主要使用路径和配置，文件内容或底层依赖升级后可能命中陈旧资产。

建议给资产目录加入 manifest。

```
schema_version  
video_sha256  
source_fps_or_pts_map  
selector_name_and_version  
group_boundaries  
canvas_order  
patch_layout
```



```
padding_policy  
position_dtype_and_shape
```

显式 manifest 可以把论文 U/G 中省略的 group 信息重新恢复，也让离线训练资产和在线推理共享同一契约。

10.6 失败回退

生产系统不应让 codec helper 失败直接吞掉整条请求。可设计分级回退。

1. 先尝试已验证的 codec backend 与热缓存。
2. Codec 不受支持或位置校验失败时，退到均匀抽帧。
3. 预算紧张时保留全局低分辨率帧和少量高分 block。
4. 不确定性高或任务需要轨迹时提高均匀帧比例。

回退结果应在元数据里明确标注，避免评测把 frame backend 混进 codec 结果。

10.7 安全和鲁棒性

恶意或损坏码流可能触发 decoder bug、异常内存、超长运行或伪造时间戳。Patched FFmpeg 是更大的攻击面，部署时应使用进程隔离、资源限额、输入时长和分辨率上限、超时、codec 白名单与补丁版本扫描。

Codec 分数也可能被转码参数操纵。同一视觉内容通过加噪或改变 GOP、QP、码率控制，可让选择地图变化。涉及监控或审核时，不能把低 bitcost 当作“无事发生”的安全证明。

第十一章 后续技术可能怎样发展

以下方向是基于论文局限和源码缺口的研究推断，不代表现有项目已经实现。

11.1 问题条件化的第二阶段选择

当前 codec 选择器不知道用户会问什么。可先用 codec 生成较大的候选池，再用文本问题或轻量视觉语义模型做第二次 rerank。静止文字、低运动目标和问题相关区域因此有机会被补回。

可验证方案如下。

- 第一阶段保留两倍目标预算的 codec candidates 和低分辨率全局帧。
- 第二阶段计算 query 与候选 patch 的轻量相似度，联合 codec 分数排序。
- 在 JumpScore、OCR、静态目标问答和普通长视频 QA 上同时比较。
- 若只提升静态任务却损害短事件召回，说明融合权重需要按任务动态调节。

11.2 可学习的率失真式 token 控制器

把视频编码的 $D + \lambda R$ 思路改写 to 视觉任务中。R 变成 token 或延迟成本，D 变成任务损失或证据缺失。选择器学习在给定预算下停止，而不是固定 K 或手工 readiness threshold。

训练可以使用 straight-through estimator、Gumbel Top-K 或离线强化学习。必须加入硬预算和回退约束，避免控制器为追求平均分数在少数视频上超支。

11.3 轨迹感知的混合预算

Global Top-K 适合短暂峰值，容易丢掉连续低强度轨迹。可以把预算拆成事件块、均匀哨兵帧和轨迹连接块。运动方向一致的邻接 patch 获得连接奖励，让被选证据形成短轨迹，而非孤立峰值。

反证指标应使用 DAVIS、MeViS、dense trajectory 和未来预测。若事件检索提高但轨迹连通率下降，混合策略仍不合格。

11.4 显式 group schema

LLaVA release 依赖每四 canvas 的隐式顺序，Mage 每 canvas 又独立视觉窗口。后续接口可显式传 `group_ids`、`group_offsets`、`canvas_ids` 与 `padding_mask`，让 attention topology 不再靠目录排序推断。

一个通用 token record 至少包含

```
[source_t, source_h, source_w, canvas_id, packed_h, packed_w, group_id, valid]
```

训练、离线缓存和在线 processor 共用带版本的 schema，能减少 silent misalignment。

11.5 跨 codec 校准

HEVC CTU bitcost、H.264 MB bitcost、VP9 SB bitcost 和 DCVC 概率码长的数值尺度不同。分位数归一能缓解，却会抹去绝对难度。可学习一个 codec-conditioned calibrator，把局部成本映射到统一的事件概率或 token utility。

评测应跨编码器、preset、QP、GOP 和转码次数，报告同一视频选择集合的 Jaccard、事件召回和最终任务分数。

11.6 真正增量的 streaming runtime

Mage 的公开 gate 网络已经具备因果状态基础，公开 demo 尚未暴露持久状态。下一步需要把 decoder、selector、视觉塔和 gate 做成逐段 API。

- Decoder 保留必要 DPB，避免每段从关键帧重新扫完整文件。
- Selector 持久维护 readiness 与逻辑组状态。
- ViT 缓存最近视觉窗口，gate 持久维护 Mamba state。
- LLM 生成明确使用最近 N 段，并有内存淘汰与时间戳一致性。
- 用户 query 插入时可选择只读缓存、回看磁盘或提高未来采样预算。

评价重点应包括流到达延迟、状态内存、每秒能耗、漏触发、误触发和回答质量。

11.7 解码到 GPU 的低拷贝路径

当前 canvas 方案经常经历解码 BGR、NumPy、RGB、JPEG/PNG、PIL、image processor 和 GPU 上传。它换来了兼容性，也产生多次拷贝和潜在 JPEG 损失。

后续可把被选 patch 写入 pinned memory 或 GPU tensor，位置表与像素一起批量上传。更进一步可在硬件解码器的 surface 上执行 patch gather。优化必须保留与 canvas 路径逐 patch 等价的回归测试。

11.8 神经 codec 与视觉模型联合设计

DCVC 路径证明概率码长可替代传统 bitcost。未来可让 codec 的概率模型同时服务重建和视觉 token utility。一个 latent 对像素重建不贵，却可能对文字或小目标识别关键，联合目标可学习这种差异。

需要防止训练变成只适配某个神经 codec。跨 codec 推理、冻结 selector 迁移和真实压缩率仍应作为约束。

11.9 音视频联合事件门控

许多流式事件先在音频出现，例如口令、撞击、欢呼和警报。只看视觉 codec 会晚触发或漏触发。Mage 论文也把原生音视频流列为未来方向。可让音频码率变化、声学事件和视觉 codec window 共同进入 gate，再由模态不确定性控制 speak 阈值。

11.10 不确定性与安全回退

选择器可以输出覆盖置信度。若候选分数分布过平、码流损坏、codec 未见或问题与静态细节相关，系统自动提高 dense anchor 或 uniform frame 预算。回答时还可记录哪些时间段从未被视觉模型观察，避免给出过度确定结论。

11.11 一个可落地的近期路线图

阶段	可交付改进	验收标准
第一阶段	修复 CLI 枚举、metadata 后端、target canvas 语义与 schema	单元测试覆盖全部已知审计缺陷
第二阶段	显式 group tensor 与统一 asset manifest	离线训练和在线推理资产逐字节对齐
第三阶段	Query-conditioned reranker 与 uniform fallback	短事件、OCR、轨迹任务无明显单边退化
第四阶段	持久 streaming state 和最近 N 段生成	逐段结果与离线因果基线一致，延迟可控
第五阶段	跨 codec 校准和 GPU 低拷贝路径	多 codec 鲁棒性稳定，端到端收益可复现

第十二章 结论

传统视频压缩器已经把世界分成可预测背景和需要额外比特解释的变化。OneVision-Encoder 把这项结构变成视觉 token 分配原则，内容仍是 RGB patch，源坐标通过 3D RoPE 保留。codec-video-prep 把 patched decoder、动态分组、2×2 Top-K、canvas 和位置数组做成可调用基础设施。LLaVA-OneVision-2 把稀疏视觉接到统一 Qwen3 MLLM，并用时间戳和局部窗口处理长视频。Mage-ViT 与 Mage-VL 再扩展到 DCVC 概率码长和主动流式门控。

源码审计同时说明，研究思想和可运行 release 之间存在多处漂移。OneVision 的 global Top-K、GOP 和训练损失与方法文字不完全一致。LLaVA 论文的 MV、residual 与 P/B 配额自适应 GOP 没有完整出现在当前 public main。Mage 论文的 HEVC 显式双信号选择器和真正在线滑窗系统也没有作为完整代码发布。准确使用这些项目，需要把论文、预处理版本、remote-code processor 和模型张量契约一起记录。

这条路线最值得延续的部分，是把 codec 当成便宜的第一阶段先验，再让任务语义、全局回退和可学习预算补足它的盲区。长期目标是让长视频系统摆脱对单一压缩格式的依赖，在有限计算里知道何时该细看、哪里要保留、何时应该开口。

附录 A 术语速查

术语	通俗解释
Codec	把视频编码成码流并从码流重建视频的一组规则与实现
Container	MP4 等封装层，保存音视频流、时间戳和索引
I/P/B	不同预测依赖的图片或 slice 类型
IDR / CRA	随机访问与解码刷新相关图片类型
GOP	编码预测结构或预处理逻辑组，必须看上下文
MV	编码块从参考图取预测区域的位移
Residual	原始块减去预测块后剩下的像素差
CTU / MB / SB	HEVC、H.264、VP9 的粗编码块单位
CABAC	结合上下文概率的二进制算术熵编码
Bitcost	一段编码语法实际或估计消耗的比特
Patch	ViT 的固定像素小块，例如 14×14 或 16×16
2×2 merge	四个相邻 patch 合成一个下游视觉 token
Canvas	把多个来源时刻的 RGB patch 紧凑拼成的规则图片
3D RoPE	用时间、高度、宽度坐标旋转 Q/K 的位置编码
Readiness	根据重要性、时间覆盖与边际收益决定逻辑组何时结束
DPB	解码画面缓冲，保存后续预测所需重建参考
EPFE	Mage 论文中的事件保留特征提取器

附录 B 仓库版本与关键入口

B.1 OneVision-Encoder

- 官方仓库 github.com/EvolvingLMMs-Lab/OneVision-Encoder
- 审计提交 c2cd3e0d85c7c44d1b1b1bad5317ceb6ff6fe93a
- 模型配置
onevision_encoder/configuration_onevision_encoder.py::OneVisionEncoderConfig
- 3D RoPE
onevision_encoder/modeling_onevision_encoder.py::VideoRotaryEmbeddingSplit466
- 模型前向 同文件 OneVisionEncoderModel.forward
- 训练主链 training/train.py::main
- Codec loader dataloader/data_decord_codec.py::VideoExternalSource
- LLaVA Stage 1/2 llava_next/Compressed_Video_Reader/tool/stage1.py、stage2.py

B.2 codec-video-prep

- 官方仓库 github.com/YunyaoYan/codec-video-prep
- 审计提交 77e8e91c11bb2fd520701e49465c9001f6c5b8ad
- 包声明版本 0.2.5
- 公开入口 src/codec_video_prep/api.py::run_preinfer
- 核心流水线 codec_selector/core/pipeline.py::run_bitcost_readiness
- Readiness codec_selector/plugins/groupers/readiness.py::build_readiness_groups
- Selector
codec_selector/plugins/selectors/topk_2x2_bitcost.py::process_group_topk_2x2
- Packer codec_selector/codec_patch_gop/patch_utils.py::pack_patches_to_canvases
- LLaVA 适配
src/codec_video_prep/integrations/llava_onevision.py::prepare_llava_onevision_inputs

B.3 LLaVA-OneVision-2

- 官方仓库 github.com/EvolvingLMMs-Lab/LLaVA-OneVision-2
- 主仓审计提交 5bede6462a0b321206c14a5982ace5c0455abd90
- HF remote code 审计提交 802f16c8346a062fef7ddf92b2bd64a770a50a1a
- Imms-eval 分支提交 3997a60cb8e79d9341ac1e4a286f0bb739bcc779
- Codec processor codec_video_processing_llava_onevision2.py::process_codec_video
- 总 processor processing_llava_onevision2.py::LlavaOnevision2Processor.__call__
- 视觉前向 modeling_llava_onevision2.py 的 vision model forward
- 训练数据入口 主仓 qwen2vl_task_encoder.py::process_sft_qa

B.4 Mage

- 官方仓库 github.com/microsoft/Mage
- GitHub 审计提交 76bec2bb3818863f470de7e867c2dc7f1d0bfd83
- HF Mage-VL 审计提交 d88b153285f1633a61b2f693c59c8576693af185
- HF Mage-ViT 审计提交 de3bd807f06fc9e8045540041dad3bf429a1eae7
- Traditional 入口 `codec_video_processing_mage_vl.py::_run_cv_preinfer`
- 视觉与 Qwen3 `modeling_mage_vl.py`
- Gate `streammind_gate.py`
- Streaming demo GitHub `mage_vl/inference_streaming.py`
- DCVC `neural_codec/dcvc_readiness_gen.py`、`dcvc_rt_engine.py`

附录 C 关键审计发现清单

编号	发现	影响
C1	codec-video-prep CLI 暴露 pkt_peak，插件识别 pkt_size_peak	所谓峰值采样会静默变成 fps 路径
C2	README 的 src_patch_position 格式与代码 N×3 不一致	下游解析可能错列或错 shape
C3	meta.json 的 decode_backend 被硬编码	性能归因可能使用错误后端
C4	LLaVA 适配器 target_canvas 不形成固定输出约束	实际 canvas 数与名称不一致
C5	OneVision MV+res 离线 step3 有确定性变量与计数 bug	公开脚本不能原样生成有效训练索引
C6	OneVision 64 帧 loader 的等号边界可能导致 RGB 与 visidx 时间错配	张量合法但源位置可能错误
C7	OneVision 论文 logistic loss，代码为多路 PartialFC softmax	训练目标不可按公式直接复现
C8	LLaVA 论文 adaptive GOP 与 release readiness 不同	论文算法和当前 main 不能互换
C9	LLaVA group id 没有显式 tensor	Canvas 顺序承担隐式语义
C10	Mage 论文 HEVC MV+res，release traditional 为 bitcost	选择信号的可见实现不同
C11	Mage 默认避开真实 keyframe，完整 anchor 是逻辑组首帧	不能把公开输出一律称为 I canvas
C12	Mage streaming demo 全片预处理，生成只用当前段	论文实时状态和最近 N 段窗口未完整实现

附录 D 主要论文与资料

1. Gary J. Sullivan, Jens-Rainer Ohm, Woo-Jin Han, Thomas Wiegand. [Overview of the High Efficiency Video Coding HEVC Standard](#). IEEE TCSVT, 2012.
2. [OneVision-Encoder Codec-Aligned Sparsity as a Foundational Principle for Multimodal Intelligence](#). arXiv 2602.08683v3, 2026.
3. [LLaVA-OneVision-2 Towards Next-Generation Perceptual Intelligence](#). arXiv 2605.25979v1, 2026.
4. [Mage-VL An Efficient Codec-Native Streaming Multimodal Foundation Model](#). arXiv 2607.24904v1, 2026.
5. 用户提供的 `codec-video-prep` 技术原理与完整流程.md。本文在其流程梳理基础上重新核对审计提交，并修正 CLI 采样枚举、位置格式、后端元数据与 target canvas 等代码差异。

附录 E 一页式总结

Codec-native 视频理解可以记成五句话。

1. 视频编码器已经计算过哪里能从过去预测，哪里需要额外比特。
2. 选择器用这些信号保留完整上下文和少量变化区域，模型读取的内容仍是 RGB patch。
3. Canvas 解决规则计算，来源位置解决时空身份，2×2 block 解决 merger 对齐。
4. OneVision 建立视觉原语，codec-video-prep 工程化，LLaVA 接入 MLLM，Mage 扩到神经 codec 与事件门控。
5. 论文算法、当前公开代码和精确评测依赖存在版本漂移，复现必须记录完整调用链与资产 schema。